

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**Designing an ETL Pipeline for Analyzing Product Prices
and Status from Lazada E-commerce Data**

NGUYEN DUY ANH

anh.nd215175@sis.hust.edu.vn

Supervisor: Associate Professor Tran Viet Trung

Signature

Program: Global ICT (IT-E7)

School: Information and Communications Technology

HANOI, 06/2025

ACKNOWLEDGMENTS

Throughout the process of completing this graduation thesis, I have received valuable support and encouragement from my teachers, family, and friends.

First and foremost, I would like to express my deepest gratitude to my parents and loved ones for always being by my side, offering unwavering support throughout my studies and work. Their encouragement has been a great source of motivation for me to constantly strive for improvement and to complete this thesis.

I would like to extend my sincere thanks to Dr. Tran Viet Trung, who provided invaluable guidance and dedicated support during the development of this thesis. Thanks to his thoughtful advice and mentorship, I was able to overcome many challenges and successfully complete this project.

I am also grateful to my friends and lecturers at Hanoi University of Science and Technology for accompanying me in both academic and personal growth, helping me gain knowledge and develop myself throughout my university journey.

During the implementation of this thesis, due to time constraints and limited experience, I am aware that there may still be shortcomings. I sincerely hope to receive feedback and suggestions from my professors and peers to further improve and refine this work.

Thank you very much!

ABSTRACT

Since its emergence in 2009, e-commerce has steadily gained a critical role in the global economy. The rapid expansion of the e-commerce market has made it increasingly difficult for businesses to keep pace with market dynamics. With the explosive growth of data volume, there is a pressing need for tools that assist business owners and product vendors in discovering market trends and making informed decisions.

Currently, many existing data analysis solutions for e-commerce are still limited in their ability to handle real-time data processing and multi-source integration. To address these challenges, this project aims to design and implement a real-time e-commerce data analytics system. The system leverages modern big data technologies including Apache Kafka, Apache Hadoop, Apache NiFi, Apache Spark, Cassandra, and PostgreSQL, following the Lambda architecture to enhance real-time processing capabilities. Data is collected directly from the Lazada API, initially stored in PostgreSQL as a staging area, then batch-processed by Apache Spark. The processed data is concurrently pushed to Kafka for real-time streaming and HDFS for raw data storage. After further processing, results are stored in Cassandra as a serving layer. To support observability and system health monitoring, Prometheus and Grafana are integrated for metrics collection and visualization. The entire system is containerized using Docker and orchestrated via Docker Compose, enabling easy deployment, scaling, and maintainability.

The main contribution of this project is the development of a scalable, real-time e-commerce data analytics pipeline capable of handling data from multiple shops and delivering actionable insights through a user-friendly dashboard. The final outcome includes a fully functional data pipeline that can ingest, process, and visualize e-commerce data in real-time, accompanied by detailed charts and analytical tables to help users monitor and explore key market trends effectively.

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.1.1 Objectives and Scope of the Graduation Thesis.....	1
1.2 Tentative solution	2
1.3 Thesis organization.....	4
CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS.....	5
2.1 Overview of E-commerce Data	5
2.1.1 Introduction to E-commerce and Lazada	5
2.1.2 Market Survey	6
2.1.3 Characteristics of e-commerce data.....	6
2.1.4 Key Metrics to Track.....	7
CHAPTER 3. METHODOLOGY.....	9
3.1 Functional Overview.....	9
3.1.1 Functional Requirements	9
3.1.2 Non-functional requirement	9
3.2 Apache Kafka – Data Ingestion Technology	10
3.3 Apache Spark – Data Processing Technology	12
3.4 Apache Cassandra – Data Storage Technology	13
3.5 Hadoop Distributed File System (HDFS) – Architecture and Components	14
3.6 Apache NiFi - Data Automation Technology	16
3.7 Docker - Application Deployment Technology	18
3.8 Visualization – Prometheus, Grafana, and Superset.....	20

CHAPTER 4. SYSTEM DESIGN AND IMPLEMENTATION	21
4.1 Architecture selection	21
4.1.1 Lambda Architecture	21
4.2 System Architecture Design.....	23
4.3 System Implementation	24
4.3.1 Data Ingestion Layer	24
4.3.2 Batch Layer	28
4.3.3 Data Processing and Storage Layer	35
4.3.4 Data Serving and Visualization Layer	41
CHAPTER 5. SYSTEM DEPLOYMENT AND EXPERIMENTAL RESULTS	
47	
5.1 System Deployment	47
5.1.1 Deployment Environment	47
5.1.2 Deployed Containers	48
5.1.3 Network Configuration	51
5.1.4 Volume Management.....	51
5.1.5 Deployment Features	51
5.2 Experimental Results	51
5.2.1 Data Collection Experiment	51
5.2.2 Data Processing and Storage Experiment	54
5.2.3 Data Visualization Experiment	54
5.2.4 System Monitoring Experiment.....	56
5.3 System Evaluation	59
CHAPTER 6. CONCLUSION AND FUTURE WORK	62
6.1 Conclusion.....	62

6.2 Future Work	63
6.3 Current System Issues and Priorities.....	64
REFERENCE	65

LIST OF FIGURES

Figure 3.1	Architecture of Apache Kafka [2]	10
Figure 3.2	Architecture of Apache Spark [5]	12
Figure 3.3	Architecture of HDFS [8]	15
Figure 3.4	Architecture of Apache Nifi [10]	17
Figure 3.5	Architecture of Docker [12]	18
Figure 4.1	Lambda Architecture [17]	21
Figure 4.2	System architecture	23
Figure 5.1	Information about containers	47
Figure 5.2	Spark job tracking interface	53
Figure 5.3	Data Collection into Kafka	53
Figure 5.4	Data Collection into Hadoop	53
Figure 5.5	Data Processing and Storage	54
Figure 5.6	Experimental Results: Data Processing and Storage	54
Figure 5.7	Store overview	55
Figure 5.8	Pricing and promotion analysis	55
Figure 5.9	Inventory and supply analysis	56
Figure 5.10	Cassandra Metrics Monitoring	57
Figure 5.11	Kafka Monitoring Dashboard	57
Figure 5.12	Spark Monitoring Dashboard	58
Figure 5.13	Prometheus alert rules	59

LIST OF TABLES

Bảng 4.1	Description of product information	26
Bảng 4.2	Description of SKU information	26
Bảng 4.3	Basic product information	35
Bảng 4.4	Variant-level information	35
Bảng 4.5	Product image metadata	36
Bảng 4.6	Brand and product description	36
Bảng 5.1	System Testing Scenarios for Lazada Data Pipeline	60
Bảng 5.2	System Performance Measurements	60

LIST OF ABBREVIATIONS

Abriviation	Full Expression
API	Application Programming Interface – A set of definitions and protocols for building and integrating application software
CSV	Comma-Separated Values – A simple file format used to store tabular data
DAG	Directed Acyclic Graph – A graph structure with directed edges and no cycles
DB	Database – An organized collection of structured data
ELK	Elasticsearch, Logstash, Kibana – A set of tools for searching, analyzing, and visualizing log data in real time
EMA	Exponential Moving Average – A type of moving average that gives more weight to recent data points
ETL	Extract, Transform, Load – A process for collecting, transforming, and storing data
HTTP	HyperText Transfer Protocol – A protocol used for transmitting hypermedia documents on the web
IT	Information Technology – The use of computers to store, retrieve, transmit, and manipulate data
JSON	JavaScript Object Notation – A lightweight data-interchange format, easy for humans to read and write

Abriviation	Full Expression
KPI	Key Performance Indicator – A measurable value that demonstrates how effectively a company is achieving key objectives
MAE	Mean Absolute Error – The average of absolute differences between predictions and actual values
ML	Machine Learning – A field of AI focused on algorithms that improve through experience
OLAP	Online Analytical Processing – Tools that allow users to analyze
RMSE	Root Mean Square Error – A standard way to measure the error of a model in predicting quantitative data
RSI	Relative Strength Index – A momentum oscillator used in technical analysis
SLA	Service Level Agreement – A contract that defines the level of service expected from a provider
SMA	Simple Moving Average – A technical indicator that calculates the average of a selected range of prices
SQL	Structured Query Language – A language used to communicate with relational databases
Student	Undergraduate Student
Thesis	Graduation Thesis
TTL	Time To Live – A mechanism that limits the lifespan or lifetime of data in a network

Abriviation	Full Expression
YAML	YAML Ain't Markup Language – A human-friendly data serialization standard often used in configuration files

CHAPTER 1. INTRODUCTION

1.1 Motivation

In the era of digital transformation, the explosion of data—commonly referred to as “Big Data”—has reshaped how businesses operate and make decisions. This term describes datasets that are too large, fast, or complex for traditional data processing methods to handle. In response, modern big data architectures and technologies have emerged, enabling businesses to extract actionable insights from raw data.

Among industries benefiting the most from big data is e-commerce. Platforms such as Amazon, Shopee, and Lazada have revolutionized the way consumers shop online. With billions of transactions occurring daily, these platforms generate massive volumes of data—including product listings, customer behavior, pricing trends, and inventory updates. This information, if harnessed properly, can support real-time business intelligence, enhance customer experience, and improve operational efficiency.

However, existing data analytics solutions for e-commerce face significant challenges. Commercial tools are often expensive and complex, requiring specialized infrastructure and technical teams. Open-source alternatives may lack features such as real-time streaming or user-friendly dashboards. Small and medium-sized businesses, in particular, often lack access to scalable, integrated solutions for data processing and analytics.

This graduation thesis is motivated by the need to address those gaps. It aims to design and implement a cost-effective, end-to-end data pipeline system tailored to Lazada, one of Southeast Asia’s leading e-commerce platforms. The proposed system leverages open-source technologies—such as Apache NiFi, Kafka, Spark, Cassandra, PostgreSQL, Superset, and Docker—to collect, transform, store, and visualize Lazada product data in both batch and real-time modes. The system is designed to be scalable, fault-tolerant, and accessible for non-technical users

1.1.1 Objectives and Scope of the Graduation Thesis

The main objective of this thesis is to build a full-stack data pipeline that automates the ingestion, processing, storage, and visualization of product data from the Lazada e-commerce platform. The project focuses on both real-time and batch data processing while providing a scalable architecture for monitoring and analytics.

The specific goals are:

- **Automated Data Ingestion:** Develop a Python-based service to collect product

metadata (e.g., item ID, name, SKUs, price, quantity, category) from the Lazada Open Platform API at scheduled intervals.

- **Streaming Architecture:** Use Apache Kafka to decouple data producers and consumers, ensuring flexible and fault-tolerant data transmission.
- **Real-time and Batch Processing:** Implement Apache Spark Structured Streaming to process incoming Kafka data. Support both immediate real-time updates and scheduled batch jobs.
- **Multistorage Strategy:**
 - **PostgreSQL** is used as a staging area during the initial data collection phase, where cleaned and normalized records are stored for reusability and ingestion into Kafka.
 - **Apache Cassandra** is used as the primary serving layer for analytics dashboards, optimized for high availability and low-latency data access.
 - **HDFS** stores archived raw batch data in Parquet format for scalable storage and historical analysis.
- **Monitoring and Visualization:**
 - Use **Prometheus** to track system health and component performance, including metrics from Kafka, Spark, and Cassandra.
 - Use **Grafana** to visualize system performance and anomaly alerts.
 - Use **Apache Superset** to provide business users with interactive dashboards and visual analytics of product data.
- **Containerized Deployment:** Deploy all services via Docker Compose to ensure consistent development and production environments, simplify scaling, and enhance maintainability.

This thesis is application-driven and serves as a prototype for building scalable, modular data systems tailored to e-commerce analytics.

1.2 Tentative solution

Based on the system goals and requirements presented in Section 0.2, this project adopts the Lambda Architecture as the foundational design pattern. The Lambda model enables seamless support for both batch and real-time data processing, offering scalability, fault-tolerance, and adaptability for large and fast-changing data sources like e-commerce platforms.

The proposed solution is structured into the following core components:

- **Data Ingestion Layer:** Product data from the Lazada Open Platform API is collected through a custom Python application. This application runs on a scheduled basis and sends the extracted data to an Apache Kafka message broker for decoupled transmission.
- **Processing Layer:** Apache Spark Structured Streaming is used to consume the Kafka topics and apply necessary transformations. The architecture supports both stream (real-time) and micro-batch modes, depending on the data freshness requirements.
- **Storage Layer:**
 - **PostgreSQL:** Stores normalized and structured data for querying, reporting, and historical analysis.
 - **Cassandra:** Used as a high-performance time-series store to support low-latency data access in dashboards.
 - **HDFS (Hadoop Distributed File System):** Archives raw batch data in Parquet format for scalable storage and future machine learning applications.
- **Visualization and Monitoring Layer:**
 - **Apache Superset:** Provides end users with interactive dashboards to explore product data and extract insights.
 - **Prometheus:** Monitors system metrics and service health, including Kafka throughput, Spark job status, and database performance.
 - **Grafana:** Offers real-time visualization of system behavior and historical trends.
- **Deployment Strategy:** All services are containerized using Docker and orchestrated via Docker Compose. This approach simplifies configuration, supports reproducibility, and facilitates local or cloud deployment.

In summary, the proposed architecture delivers a modular and extensible big data pipeline tailored to the analytics needs of e-commerce platforms. It integrates modern open-source tools to support end-to-end data flow, from ingestion to actionable visualization, and forms a solid foundation for further extensions such as predictive analytics, demand forecasting, or multi-source data integration.

1.3 Thesis organization

The remainder of this thesis is organized as follows:

Chapter 2 provides an overview of the current e-commerce landscape, including key development trends and characteristics of consumer behavior in the digital environment. In addition, I review and analyze several existing e-commerce data analytics systems that have been deployed in practice. By comparing and evaluating these systems, I identify the core requirements that the proposed system must fulfill to be practical and effective.

In Chapter 3, I introduce the technologies adopted in the development of the system and briefly explain the rationale behind each selection. These technologies include Apache Hadoop, Apache Spark, Apache NiFi, Apache Kafka, Apache Cassandra, Prometheus, Grafana, Superset, and Docker. The chosen technologies aim to ensure the system's capability to process large volumes of data, support both batch and real-time processing, and enable efficient data visualization and system monitoring.

Chapter 4 presents the analysis, design, and implementation of the system based on the Lambda architecture. In this chapter, I describe the detailed structure of each functional layer, including the data ingestion layer, the processing and storage layer, the serving layer, and the visualization layer. The chapter also discusses the deployment of the system using Docker and Docker Compose to facilitate flexible and scalable implementation.

Chapter 5 discusses the outcomes achieved after system deployment. I conduct comprehensive testing to evaluate the system's stability, performance, and real-time processing capability. The results are analyzed to assess how well the system meets the predefined requirements and to demonstrate its effectiveness in the context of e-commerce data analysis.

Finally, Chapter 6 summarizes the entire project, highlighting the main results and key contributions of the system. I also discuss existing limitations and propose potential directions for future development to further enhance system performance, expand integration capabilities, and improve real-world applicability.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

Chapter 2 provides fundamental background knowledge on the current state of the e-commerce market. It also introduces several commonly used tools for collecting and analyzing e-commerce data obtained from Lazada's API and its individual stores. In addition, this chapter presents an overview of the system requirements and outlines the ETL and data visualization architecture that I have chosen to implement in this project.

2.1 Overview of E-commerce Data

2.1.1 Introduction to E-commerce and Lazada

E-commerce refers to the business activities conducted through the internet, enabling buyers and sellers to interact and transact online without the need for face-to-face meetings. A key characteristic of e-commerce is its ability to operate 24/7, regardless of geographical boundaries, providing users with a highly convenient and accessible shopping experience through digital platforms.

Lazada, founded in 2012, is one of the largest e-commerce platforms in Southeast Asia. Acquired by Alibaba Group in 2016, Lazada has evolved into a comprehensive digital commerce ecosystem that hosts millions of products from both local and international sellers. The platform offers an integrated solution for online selling, including warehouse management, logistics, and payment systems.

The current e-commerce market consists of various business models. Common types include:

- **B2C (Business-to-Consumer):** Platforms such as Lazada and Shopee, where businesses sell directly to end consumers.
- **B2B (Business-to-Business):** Platforms like Alibaba, where transactions occur between companies.
- **C2C (Consumer-to-Consumer):** Platforms such as online classified ads where individual users buy and sell directly to each other.

Each model employs unique technologies and operational approaches to serve their specific customer base. The rapid growth of e-commerce has not only transformed traditional shopping behaviors but also given rise to a complete digital ecosystem encompassing logistics, financial technology (fintech), and digital marketing.

2.1.2 Market Survey

The Southeast Asian e-commerce market has experienced exponential growth since Lazada's launch in 2012. According to Statista (2023), the region's e-commerce revenue is expected to exceed 100 billion USD by 2025, driven by increasing internet penetration and mobile commerce adoption. Platforms like Shopee, Tiki, and Lazada continue to compete aggressively through pricing strategies, logistics optimization, and personalized recommendations.

While this digital expansion opens new opportunities, it also presents significant analytical challenges. With millions of daily transactions and dynamic pricing mechanisms, sellers and businesses must analyze vast volumes of product and user data to make informed decisions. For instance, a single Lazada product category (e.g., electronics) may contain thousands of SKUs with frequent price and stock fluctuations, making manual tracking infeasible.

In response to these demands, several global tools have emerged to support e-commerce analytics. Solutions like Jungle Scout, Helium 10, and Sellics offer features such as product tracking, sales estimates, keyword research, and profitability analysis. These tools serve Amazon sellers primarily, but their architecture illustrates key functionalities that can be applied to other marketplaces like Lazada. However, many of these platforms are either paid (e.g., Helium 10's Diamond plan costs over \$200/month) or lack localization for Southeast Asian markets, making them less accessible for small sellers in the region.

Additionally, while these tools provide intuitive dashboards and predictive insights, they often require user onboarding and technical understanding, limiting usability for non-technical users. These practical gaps motivate the development of a custom, open-source solution tailored to Lazada, focusing on affordability, real-time tracking, and ease of use. The system proposed in this thesis is designed to bridge this gap by offering a modular and cost-effective architecture for localized e-commerce analytics.

2.1.3 Characteristics of e-commerce data

E-commerce data is inherently **large-scale**, **dynamic**, and **heterogeneous**, posing multiple challenges in the process of data collection, transformation, and analysis. The following characteristics are commonly observed:

- **High Volume:** Thousands to millions of products are listed across various categories and sellers. Each product may have multiple attributes such as pricing, inventory, ratings, etc.

- **High Velocity:** Product-related data, especially price, discount, and stock levels, are frequently updated—often multiple times a day. This necessitates near real-time ingestion and processing.
- **Multi-source and Unstructured:** Data originates from multiple sources, including APIs, web scraping (HTML), seller portals, and third-party platforms. These sources often have inconsistent formats and schemas.
- **Temporal Nature:** Most e-commerce data points (e.g., price) have value over time, making them suitable for *time-series analysis* to track trends, detect anomalies, or support decision-making.
- **Semi-structured Schema:** Some fields are structured (e.g., product ID, category), while others may vary or be missing depending on the source (e.g., product ratings, seller info), requiring flexible schema handling.

Given these properties, designing an ETL and data pipeline system for e-commerce data requires special attention to **scalability**, **schema flexibility**, and **data freshness**.

2.1.4 Key Metrics to Track

In the context of analyzing e-commerce data from Lazada, several key metrics are crucial to understanding product behavior, market trends, and business performance. These include:

- **Product Price (price):** This is the current listed price of a product on Lazada. It reflects the market value at a given time, influenced by the seller's pricing strategy as well as supply and demand dynamics. Product price serves as a fundamental indicator for evaluating the product's real-time market value.
- **Gross Merchandise Value (GMV):** GMV represents the total monetary value of all products sold on the platform. It is calculated using the formula:

$$\text{GMV} = \text{Product Price} \times \text{Total Quantity Sold}$$

Product categories with high GMV typically hold substantial market influence, and even small price fluctuations can significantly impact the broader market.

- **Transaction Volume:** This refers to the total number of completed orders within a specific time frame. It indicates the product's liquidity and level of activity on the platform. A high transaction volume implies strong demand and product popularity.
- **Performance Indicators:** These are calculated based on a combination of data

points such as product price, sales volume, and customer ratings. They are used to predict market trends and identify optimal periods for adjusting business strategies. Performance indicators assist sellers in making informed decisions to enhance operational effectiveness.

By monitoring and analyzing these metrics, the system enables a deeper understanding of product performance, market dynamics, and competitive positioning in the Lazada ecosystem.

CHAPTER 3. METHODOLOGY

This chapter presents the methodology and technologies used to develop the data pipeline system for e-commerce product analysis. It introduces the overall approach to collecting, processing, storing, and visualizing data, along with a detailed description of the key tools and frameworks employed in each stage of the system. The chosen technologies—such as Apache Kafka, Apache Spark, Cassandra, PostgreSQL, and Apache Superset—are selected based on their scalability, real-time processing capabilities, and compatibility with large-scale data environments.

3.1 Functional Overview

Based on the analysis of the e-commerce market and the capabilities of prominent monitoring and analytics tools, this project aims to build a system for collecting, transforming, storing, and visualizing Lazada e-commerce transaction data with the following requirements:

3.1.1 Functional Requirements

The system must be capable of collecting data from the Lazada API and storing it in a structured format. The collected data includes product information (such as name, price, description, and images), sales data (number of units sold, revenue, ratings), and performance indicators.

The system should also be able to update data in real-time to ensure the accuracy of product information and reflect current market trends.

For visualization, the system must provide an intuitive dashboard that displays business metrics in the form of tables and charts. These metrics and visualizations should be logically organized according to business needs, allowing users to effectively monitor sales performance and market dynamics.

3.1.2 Non-functional requirement

The system must operate reliably and support periodic automatic data updates to ensure high availability and trustworthiness for end-users. The user interface should be user-friendly, intuitive, and visually appealing, enabling users to easily grasp market activity and interact with the system without requiring technical expertise.

The system should respond quickly to user requests, minimizing latency and errors during interaction.

Data storage must be distributed and fault-tolerant to maintain continuous operation

and prevent data loss in the event of system failures. Moreover, the system should support rapid and efficient data recovery to preserve data integrity and ensure availability under all circumstances.

3.2 Apache Kafka – Data Ingestion Technology

Apache Kafka [1] is a real-time data streaming system initially developed by LinkedIn and currently maintained as an open-source project under the Apache Software Foundation. With the ability to process hundreds of thousands of records per second and deliver low latency, Kafka has become a widely adopted solution for systems that require large-scale, stable, and scalable data ingestion and transmission.

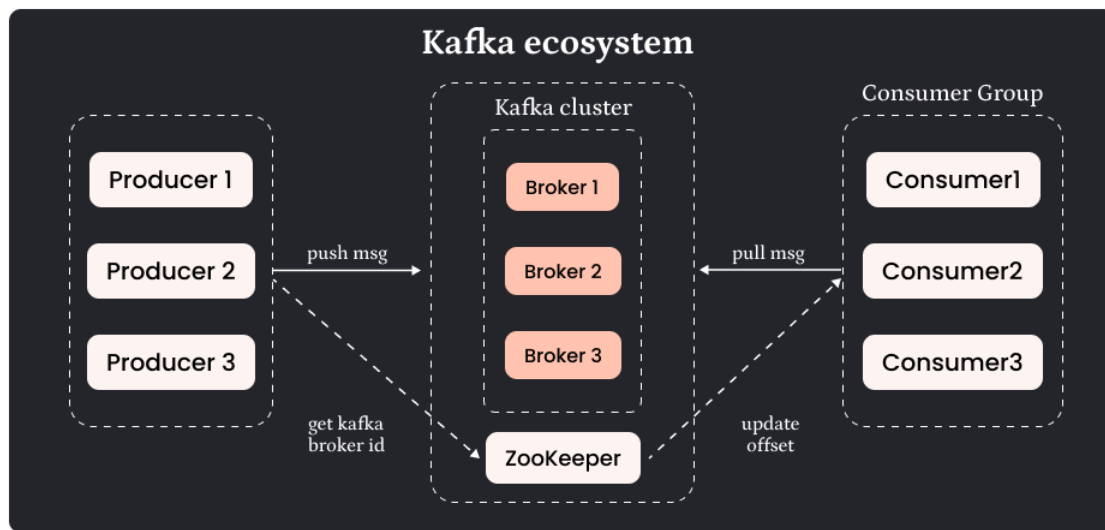


Figure 3.1: Architecture of Apache Kafka [2]

In this project, Kafka serves as the **data ingestion layer**, responsible for receiving product information crawled from the Lazada platform. The data is organized into *topics* within Kafka and transmitted as messages to downstream processing systems such as Apache Spark.

Core components of Kafka include:

- **Producer:** applications that send data into Kafka.
- **Consumer:** applications that subscribe to topics and process data.
- **Broker:** Kafka server that stores and distributes messages.
- **Topic and Partition:** organize the data stream and distribute load across brokers.
- **Offset:** the position of a record within a partition, used by consumers to track read

progress.

Connection to system requirements (see Section 2.2): The system requires real-time updates to reflect product price fluctuations and market dynamics. Kafka is well-suited for this requirement due to its ability to record data streams in real time, support parallel processing, and provide temporary buffering for downstream consumers.

Zookeeper and Kafka cluster management:

Kafka uses Zookeeper [3] to manage the distributed state and coordinate activities across the cluster. Zookeeper stores metadata about brokers, topics, and partitions. When changes occur (e.g., adding or removing brokers), Zookeeper updates the system state and notifies relevant components. This ensures consistency and high availability across the Kafka ecosystem.

Comparison with alternative technologies:

In addition to Kafka, several other platforms offer real-time data ingestion capabilities, such as Amazon Kinesis, Azure Event Hubs, and Google Cloud Pub/Sub. However, Kafka is selected for this project based on the following advantages:

- **Superior throughput:** Kafka can handle over one million events per second on a single cluster, whereas Amazon Kinesis is limited to approximately 2,000 events/second per shard.
- **High availability and fault tolerance:** Kafka is designed to remain operational even if some brokers fail. It supports data replication and persistent offset tracking, offering up to 99.99% uptime.
- **Flexibility and customization:** Kafka can be extended with tools such as Kafka Streams and Kafka Connect, and allows deep customization of deployment and operations. In contrast, cloud services often limit configurability and extensibility.
- **Cost efficiency:** Self-hosted Kafka significantly reduces operational costs compared to cloud-based services like Kinesis, which may incur 2–3 times higher expenses for the same data volume.

Given its performance, reliability, configurability, and cost advantages, Apache Kafka is the most appropriate choice for implementing the data ingestion layer in the proposed e-commerce data analysis system.

3.3 Apache Spark – Data Processing Technology

Apache Spark [4] is a unified analytics engine for large-scale data processing. It was initially developed by Matei Zaharia at UC Berkeley’s AMPLab in 2009 and open-sourced in 2010 under the BSD license. In 2013, the project was donated to the Apache Software Foundation and is now maintained under the Apache 2.0 license.

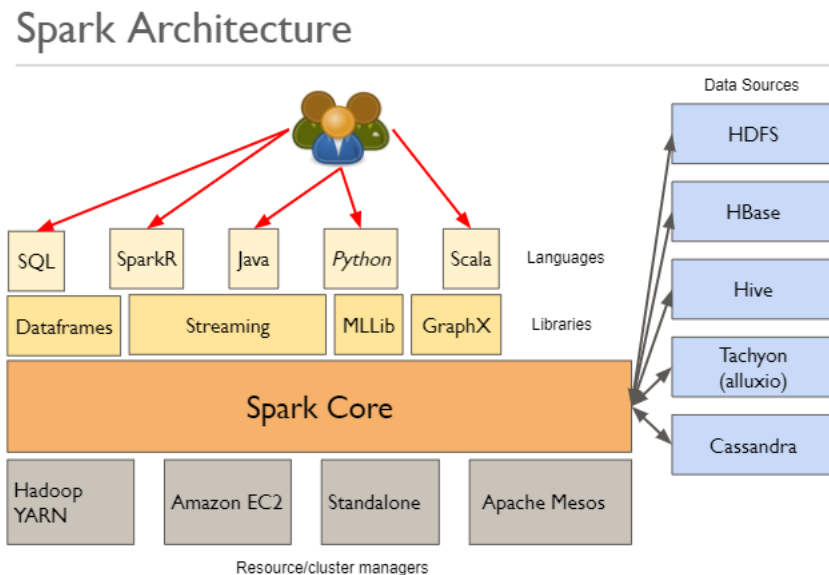


Figure 3.2: Architecture of Apache Spark [5]

Spark delivers high-performance data processing through an in-memory architecture and supports complex computations across distributed clusters. With its core abstraction—Resilient Distributed Datasets (RDDs)—Spark ensures fault tolerance and efficient computation. Additionally, Spark offers a wide range of APIs that support batch processing, real-time streaming, and machine learning.

Key libraries include Spark SQL, Spark Streaming, MLlib, and GraphX, enabling Spark to serve as a general-purpose platform for various data analytics scenarios. In this project, we focus on using Spark Core, Spark Streaming, and Spark SQL to process and transform real-time product data from Kafka into structured formats for storage and visualization.

Key components used in this project:

- **Spark Core:** Provides distributed scheduling, memory management, and RDD operations.
- **Spark Streaming:** Enables real-time processing by dividing data streams into micro-batches; integrated with Kafka.

- **Spark SQL:** Supports structured queries on diverse data formats via SQL and DataFrame APIs.

In the scope of this project, the components utilized from Apache Spark include Spark Core, Spark Streaming, and Spark SQL. While there are other big data processing technologies such as Apache Hadoop MapReduce, Apache Spark demonstrates superior capabilities due to the following advantages.

In terms of processing performance, Spark significantly outperforms MapReduce. Thanks to its in-memory architecture and features such as Resilient Distributed Datasets (RDDs), Spark is able to execute complex computations at high speed. Benchmarks have shown that Spark can run up to 100 times faster than MapReduce when working with in-memory data. This is particularly important for applications that require low-latency data processing, such as real-time analytics.

Moreover, Spark offers a diverse set of APIs that allow users to process data in multiple ways—not just batch processing like MapReduce. Libraries such as Spark SQL and Spark Streaming further extend Spark’s capabilities, making it a more flexible and comprehensive solution for large-scale data analytics.

3.4 Apache Cassandra – Data Storage Technology

Apache Cassandra [6] is a highly scalable, fault-tolerant NoSQL database designed for managing large volumes of data with low-latency access. It uses a peer-to-peer architecture with no single point of failure and distributes data across nodes using partitioning based on consistent hashing. Each node stores a portion of the data and can independently serve read/write requests, ensuring high availability and linear scalability.

In this project, Cassandra is used to store real-time product snapshots and price history data collected from Lazada. This aligns with the system’s non-functional requirement for fault tolerance and distributed storage (see Section 2.2), especially in handling high-frequency updates and time-series data.

Key concepts in Cassandra:

- **Node & Data Center:** A node is a single Cassandra server. Nodes are grouped into data centers, which can span multiple geographic regions to enhance availability.
- **Keyspace & Partitioning:** A keyspace acts as the logical container for tables and defines the replication strategy. Data is partitioned using a partition key, which determines how records are distributed across nodes.

- **CQL (Cassandra Query Language):** CQL offers SQL-like syntax, enabling familiar querying of structured data with support for CRUD operations.
- **Replication:** Cassandra supports tunable consistency with configurable replication factors, allowing data to be stored redundantly across nodes.
- **Memtable & SSTable:** New data is first stored in memory (Memtable) and later flushed to immutable on-disk structures (SSTables) for optimized read performance.

Alternative data storage technologies similar to Apache Cassandra include MongoDB and Apache HBase. However, Cassandra is chosen for this project due to several advantages over these alternatives.

While MongoDB adopts a document-based model, Cassandra follows a column-oriented model. Although this model is generally less flexible, it offers critical advantages such as stronger consistency and higher scalability. These characteristics make Cassandra particularly well-suited for applications that handle large volumes of data.

Cassandra's robust distributed and replication architecture enables it to maintain high availability even in the event of node failures, which is essential for applications with strict uptime requirements. Furthermore, Cassandra uses Cassandra Query Language (CQL), which is similar to SQL, making it more accessible and easier to use compared to MongoDB's proprietary query syntax or HBase's Java-based API.

3.5 Hadoop Distributed File System (HDFS) – Architecture and Components

Hadoop Distributed File System (HDFS) [7] is an open-source distributed file system designed to store and process massive volumes of data across clusters of commodity hardware. HDFS adopts a master-slave architecture, in which the **NameNode** functions as the master managing metadata, while **DataNodes** act as slaves responsible for storing actual data blocks. HDFS ensures high data reliability by replicating blocks across multiple DataNodes and allows parallel data access to optimize read/write performance.

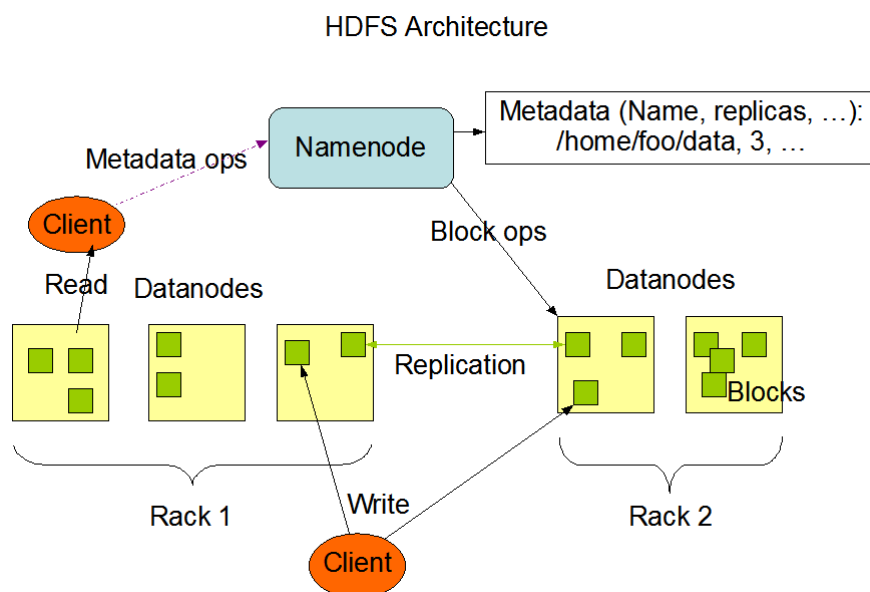


Figure 3.3: Architecture of HDFS [8]

HDFS supports horizontal scalability by allowing new DataNodes to be added seamlessly to the cluster, automatically balancing the data load among nodes. These advantages make HDFS an ideal storage solution for e-commerce data analytics systems, especially when handling large volumes of product, transaction, and user behavior data from platforms such as Lazada. As a distributed file system, HDFS also serves as a foundational layer for batch processing with Apache Spark, enabling efficient and reliable data analysis.

Core components of the HDFS architecture include:

- **NameNode:** The central component responsible for managing the entire filesystem namespace. It maintains a directory tree of all files in the system and tracks the location and replication status of each data block. Metadata is kept in memory for fast access and persisted to disk using `FsImage` and `EditLog` for durability.
- **DataNodes:** These are the physical storage nodes that store and manage data blocks. Each DataNode periodically sends heartbeat signals and block reports to the NameNode to report its status and the blocks it stores. They handle client read/write requests and participate in block replication to ensure data reliability.
- **Client:** The interface through which users interact with HDFS. For reads, the client contacts the NameNode to obtain block locations, then retrieves data directly from the nearest DataNodes. For writes, the client splits the file into blocks, coordinates with the NameNode to assign target DataNodes, and writes blocks to the first node

in the replication pipeline.

- **Block Operations:** Files are divided into fixed-size blocks (typically 128MB). Each block is replicated (default of three copies) and distributed across multiple DataNodes to ensure availability and fault tolerance. The NameNode monitors and maintains the required number of replicas.
- **Replication Pipeline:** A mechanism through which data blocks are copied across DataNodes. When a client writes a block, it is sent to the first DataNode, which then forwards it to a second, and so on. This ensures efficient and reliable replication.
- **Block Placement Policy:** This policy determines how blocks and their replicas are distributed. The algorithm takes into account network topology, node load, and rack-awareness to optimize fault tolerance and data locality.
- **Fault Tolerance:** HDFS is resilient to hardware failures. If a DataNode becomes unavailable, the NameNode detects it via missed heartbeat signals and automatically re-replicates the lost blocks to healthy nodes, ensuring consistent data availability.

Alternative distributed storage technologies include GlusterFS, Ceph, and Amazon S3. However, Hadoop Distributed File System (HDFS) is chosen for this project due to its design tailored for large-scale data processing. HDFS offers linear scalability and seamless integration with the Hadoop and Spark ecosystems, making it highly suitable for big data analytics applications. Its automatic data replication mechanism ensures high reliability and fault tolerance. Furthermore, HDFS benefits from a large user community and extensive documentation, which facilitates implementation, maintenance, and troubleshooting within enterprise environments.

3.6 Apache NiFi - Data Automation Technology

Apache NiFi [9] is an open-source data integration tool designed to automate the flow of data between systems. It provides a highly configurable web-based interface that enables users to design, monitor, and manage data pipelines with ease. NiFi supports dynamic routing, transformation, and mediation of data, making it suitable for various data integration and automation scenarios.

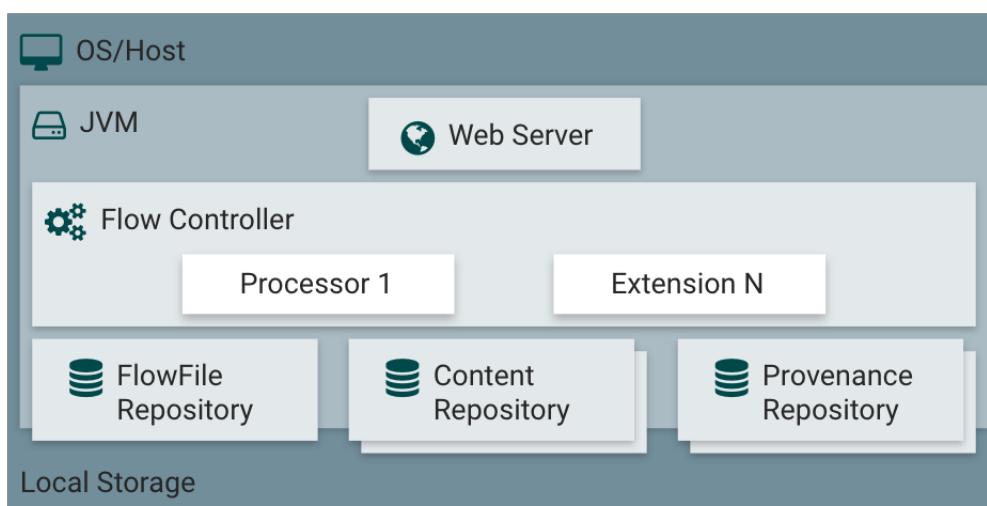


Figure 3.4: Architecture of Apache Nifi [10]

NiFi's architecture is composed of the following core components:

- **NiFi Flow Controller:** The central component that manages the scheduling and execution of data flows. It coordinates data movement and transformation across the entire system.
- **NiFi Web UI:** A graphical user interface that allows users to design and monitor data pipelines through a drag-and-drop paradigm, eliminating the need for extensive coding.
- **NiFi Processors:** These are modular units that perform specific data operations such as data ingestion, transformation, or dispatch to target systems. Processors can be configured and chained together to construct complex flows.
- **NiFi Provenance Repository:** This component tracks the lineage of data as it flows through the system. It logs detailed metadata about data origins, transformations, and destinations, supporting auditing and debugging.
- **NiFi FlowFile Repository:** Manages the state of data (FlowFiles) as they traverse the system, ensuring data integrity and recoverability in case of failures.
- **NiFi Content Repository:** Stores the actual content of FlowFiles, allowing efficient storage and processing of large volumes of data.
- **NiFi Cluster:** Supports horizontal scalability and high availability by allowing multiple NiFi nodes to operate in coordination. Clustering ensures system resilience and efficient handling of large workloads.

- **NiFi Security:** Provides robust security features including user authentication, role-based access control, and end-to-end data encryption to ensure secure data transmission.

Compared to traditional integration methods or custom ETL scripts, NiFi offers significant advantages in terms of usability, scalability, and maintainability. Similar tools in the data automation domain include Apache Airflow, StreamSets, and Talend. However, NiFi was selected for this project due to its real-time data routing capabilities, strong visual interface, built-in provenance tracking, and native integration with Kafka and HDFS - components used elsewhere in the system. These factors make NiFi a powerful and flexible tool for automating data flows in the proposed e-commerce data pipeline.

3.7 Docker - Application Deployment Technology

Docker [11] is an open-source platform designed for building, deploying, and managing applications within containerized environments. It enables developers to package applications and their dependencies into lightweight, portable containers. Each container includes everything needed to run the application—such as code, runtime, libraries, and configuration files—ensuring consistency across different deployment environments. Docker also provides powerful tools to manage the entire container lifecycle, from building and sharing to running containers, which significantly accelerates application development and deployment. By using Docker, users can easily deploy multiple isolated environments on a single host system.

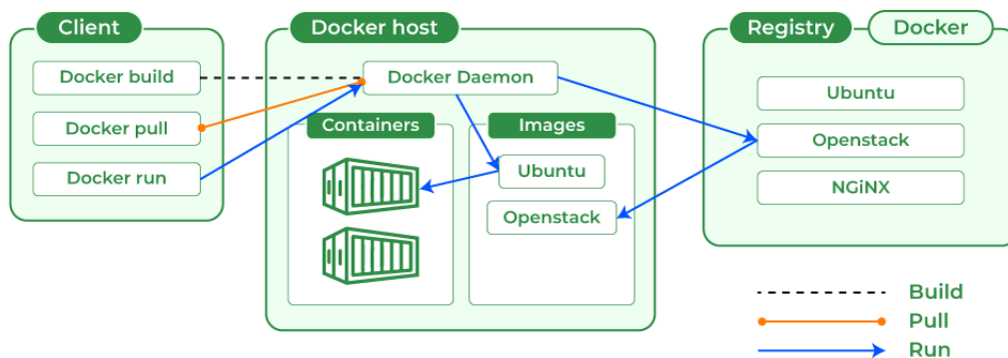


Figure 3.5: Architecture of Docker [12]

The main components of Docker architecture include:

- **Docker Daemon:** The core service running on the host machine that manages the lifecycle of containers. It listens for Docker API requests from the Docker client and performs container-related operations such as creating, starting, stopping, and

removing containers. It also manages images, networks, and volumes.

- **Docker Client:** The command-line interface (CLI) through which users interact with Docker. The client sends user commands to the Docker Daemon and displays the responses. Common tasks such as building images, running containers, and managing networks and volumes are performed via the Docker client.
- **Docker Registries:** These are repositories that store and distribute Docker images. Users can push (upload), pull (download), and share images through registries. Docker Hub is the default public registry maintained by Docker Inc., containing a large collection of pre-built and popular images.
- **Docker Images:** Immutable templates used to create Docker containers. An image includes the application code along with all necessary dependencies, such as libraries, environment variables, and configuration files. Images are built in layers, allowing for efficient reuse and sharing.
- **Docker Containers:** Running instances of Docker images. A container encapsulates an application and its environment in a lightweight, isolated execution unit. Containers can be started, stopped, moved, or deleted independently, ensuring that the application runs identically in different environments. Containers serve as the fundamental unit of deployment in Docker.
- **Docker Network:** This component enables communication between containers and external systems. Docker supports various network types such as bridge, host, and overlay networks to accommodate different communication needs.
- **Docker Volume:** A mechanism that allows persistent data to be stored outside the container's writable layer. Volumes can reside on the host machine or on remote storage services. They enable data to persist even after the container is removed, and can be shared between multiple containers, enhancing reusability and data durability.

Docker was chosen for this project due to its ability to ensure consistent environments across development and deployment stages. It simplifies containerized deployment of tools like Spark, Kafka, and Cassandra, enabling fast setup and scaling. Moreover, Docker enhances maintainability and portability, making it suitable for building flexible and reliable data pipelines.

3.8 Visualization – Prometheus, Grafana, and Superset

Prometheus, Grafana, and Superset are powerful tools for monitoring and data visualization.

Prometheus [13] is an open-source monitoring and alerting system specifically designed to collect and store real-time metrics. It operates on a pull-based model, periodically gathering metrics from configured targets via HTTP endpoints. Prometheus provides a flexible query language called *PromQL* for querying and analyzing time-series data, and it supports a wide variety of *exporters* to integrate with different systems.

Grafana [14] is an open-source platform for monitoring and visualizing metrics. It provides a user-friendly web interface that enables users to create dynamic dashboards, with support for various chart types and data panels. Grafana integrates well with multiple data sources, especially Prometheus, allowing complex queries and interactive visualization. It also features customizable alerting rules and notifications when metric thresholds are exceeded.

Superset [15] is an open-source data exploration and visualization platform designed to create interactive dashboards and reports. It offers a rich set of visualization options and supports SQL-based data exploration, making it ideal for analyzing large-scale datasets. Superset integrates seamlessly with a variety of databases and data warehouses, allowing users to generate complex visualizations and gain insights from the data.

The combination of Prometheus, Grafana, and Superset provides a comprehensive monitoring and visualization solution. Prometheus handles metric collection and storage from key system components such as Spark, PostgreSQL, and Cassandra. Grafana serves as a visual interface to monitor system performance and analyze real-time e-commerce data. Superset complements the stack with advanced data exploration and reporting capabilities, enabling users to build insightful dashboards.

Due to these advantages, Prometheus, Grafana, and Superset were selected to serve the monitoring and visualization needs of the data pipeline system in this project.

CHAPTER 4. SYSTEM DESIGN AND IMPLEMENTATION

In this chapter, I will provide a comprehensive analysis of each component of the system to address the requirements outlined in Chapter 2, by applying the technologies presented in Chapter 3.

4.1 Architecture selection

After researching and analyzing various architectural models to meet the specified requirements, I decided to design the system based on the Lambda architecture.

4.1.1 Lambda Architecture

Lambda architecture [16] is a data processing design pattern developed to handle massive quantities of data by utilizing both batch and stream processing methods. It is particularly well-suited for e-commerce analytics systems, where both historical and real-time data insights are critical.

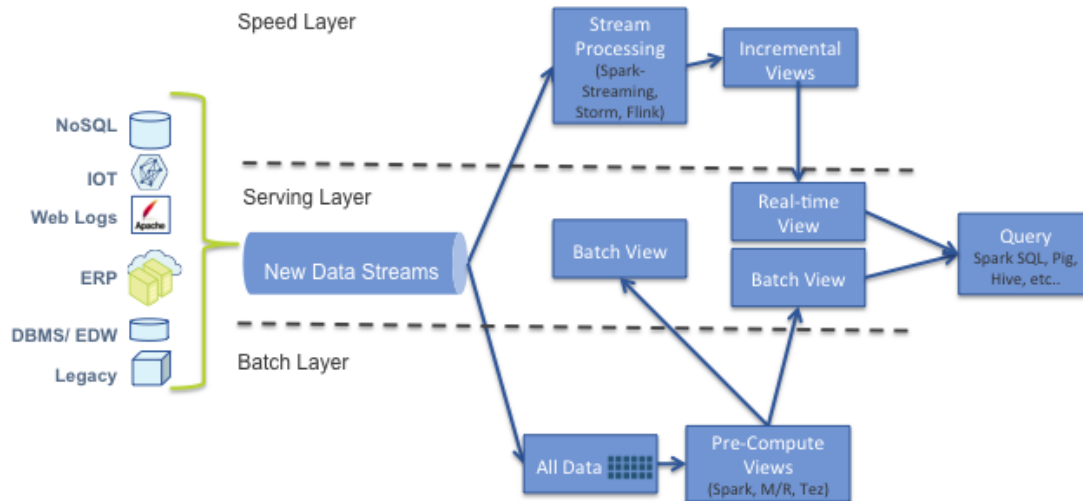


Figure 4.1: Lambda Architecture [17]

The figure above illustrates the Lambda architecture, which consists of three main layers: the Batch Layer, the Speed Layer, and the Serving Layer. Each layer plays a crucial role in the operation of the system:

- **Batch Layer:** This layer handles historical data storage and batch processing. Product data ingested through Kafka is written to Hadoop HDFS and PostgreSQL. Periodic batch jobs compute analytical results over the full dataset. These batch views are visualized through Apache Superset, enabling detailed data exploration and reporting.

- **Speed Layer:** Designed to handle real-time data, this layer uses Spark Streaming to consume data directly from Kafka. It applies transformation logic on-the-fly and writes the processed data to Apache Cassandra, allowing for low-latency access to the most recent data. This ensures the system can reflect updates almost instantly.
- **Serving Layer:** The entire system is monitored by Prometheus, which collects metrics from Spark, Kafka, and other components. These metrics are visualized using Grafana, providing real-time insights into system health and performance. Simultaneously, Superset serves as the dashboard interface, connecting to the batch output in PostgreSQL.

Advantages of Lambda Architecture:

- **High accuracy:** Batch processing ensures correctness using the full historical dataset.
- **Real-time updates:** Speed layer allows the system to quickly reflect new data.
- **Fault tolerance:** Redundant data storage across layers ensures high availability.
- **Separation of concerns:** Each layer can be independently optimized.

Disadvantages of Lambda Architecture:

- **Operational complexity:** Maintaining two parallel processing paths increases system complexity.
- **Resource intensive:** Running multiple processing engines requires significant infrastructure.
- **Latency in batch layer:** Full data recomputation can be time-consuming.
- **Synchronization overhead:** Ensuring consistency between batch and speed layers can be challenging.

In conclusion, Lambda architecture provides a balanced solution combining the accuracy of batch processing with the immediacy of real-time stream processing. For this project, it was selected as the architectural foundation due to its ability to meet the diverse requirements of an e-commerce analytics platform.

4.2 System Architecture Design

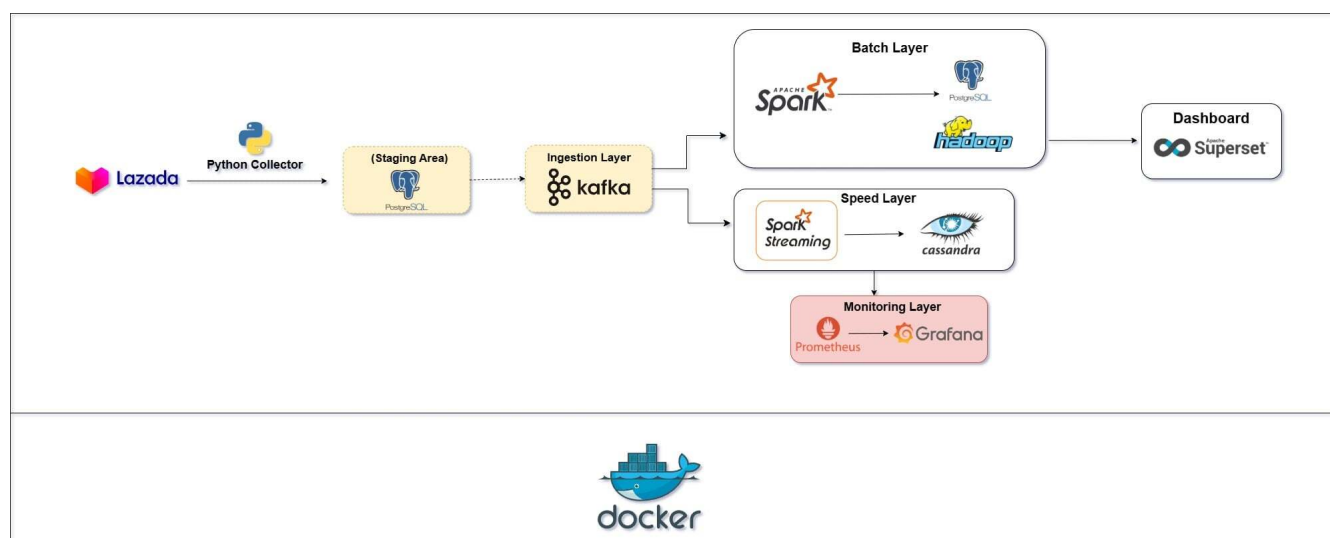


Figure 4.2: System architecture

The system is designed following the Lambda architecture paradigm, which supports both batch and real-time data processing through distinct, interrelated layers. The architecture is composed of four primary layers:

- **Data Ingestion Layer:** Data is collected from the Lazada Open Platform using a custom-built Python Collector. This collector retrieves comprehensive product data such as pricing, inventory, and metadata. The data is first stored in a PostgreSQL database that serves as a staging area, allowing temporary storage and optional pre-processing before being sent to the ingestion system. From there, data is published into Apache Kafka, which acts as the central messaging and data distribution layer, enabling decoupled and scalable downstream processing.
- **Processing and Storage Layer:** This layer follows the dual-path model of Lambda architecture, where Apache Spark in the Batch Layer periodically processes data from Kafka and loads the results into PostgreSQL for structured analytics and into HDFS (Hadoop Distributed File System) for long-term storage and large-scale computation, with PostgreSQL also serving as the backend for analytical dashboards. Simultaneously, in the Speed Layer, Spark Structured Streaming consumes streaming data in near real-time as it arrives in Kafka, performs immediate transformations, and writes the results to Apache Cassandra, a distributed NoSQL database optimized for high-throughput and low-latency querying.
- **Monitoring and Visualization Layer:** The Monitoring Layer is responsible for

tracking the health and performance of the system, where Prometheus is used to scrape metrics from critical services such as Spark, Kafka, and Cassandra, and these metrics are visualized through Grafana dashboards to provide real-time insights into workloads, reliability, and system behavior. Simultaneously, processed and curated data—primarily from the PostgreSQL storage in the batch layer—is visualized using Apache Superset, which offers an interactive interface for building analytical dashboards and reports. To support deployment and scalability, all components of the system are containerized using Docker, enabling reproducible environments, simplified orchestration, and seamless scaling across development, testing, and production stages.

4.3 System Implementation

4.3.1 Data Ingestion Layer

a, Data Source

As previously mentioned, data is collected from the Lazada API—one of the leading e-commerce platforms in Southeast Asia with millions of active products and users. The collected data includes detailed product information such as: `item_id` for unique product identification, `name` for basic descriptive content, `primary_category` to classify and group products, `status` to indicate the product’s availability, and timestamps like `created_time` and `updated_time` to track the product’s lifecycle on the platform.

For each product, the system also collects detailed SKU information, including: `sku_id` to identify different product variants, `seller_sku` for inventory tracking by sellers, `shop_sku` to associate products with specific stores, `quantity` and `available` to manage inventory levels, and `price` and `special_price` to monitor pricing strategies and promotions. Additionally, the system stores comprehensive product image data (`image_url`) along with supplementary attributes for enhanced display and search functionality.

Within the scope of this project, data is collected in two modes: real-time ingestion to capture dynamic changes in pricing, inventory, and product status; and historical collection to support trend analysis and business model evaluation. The system is designed with high scalability to handle large data volumes from the Lazada platform, ensuring continuous and reliable data acquisition. The collected data serves as a foundation for in-depth analysis of e-commerce market trends, user behavior, and seller performance on the platform.

b, Data Collection

In this project, I utilize the Lazada Open Platform API to access and collect data from the Lazada platform. This API provides standardized endpoints for retrieving detailed product information, including name, description, pricing, inventory levels, seller details, images, and other attributes. Additionally, the API offers endpoints related to transactions, allowing tracking of orders, transaction statuses, and other relevant information.

Interaction with the Lazada REST API is implemented using the `requests` library in Python. The process typically starts by constructing HTTP requests (usually GET) to specific API endpoints. To perform these requests, authentication credentials—typically an API Key and Secret Key provided after registering an application with Lazada—must be included. These keys are used to generate digital signatures for each request, ensuring secure and authenticated communication between the application and Lazada’s servers.

When sending requests, parameters can be passed via the URL to filter or specify the desired data. For example, parameters may be used to retrieve a list of products within a specific category or to search for products based on keywords. The `requests` library offers convenient methods to build URLs with parameters easily.

The API responses are returned in JSON format, containing structured product or transaction data. These structures are aligned with the schema designed for both PostgreSQL and Cassandra databases.

The PostgreSQL [18] schema focuses on relational modeling of product data, with related tables representing complex relationships between entities. For example, there may be tables for products, categories, sellers, product attributes (e.g., color, size), and junction tables to capture many-to-many relationships. Each table includes columns representing different data fields with appropriate data types (e.g., `VARCHAR` for names, `TEXT` for descriptions, `DECIMAL` for prices, `INTEGER` for quantities). Primary and foreign key constraints are defined to ensure data integrity.

In contrast, the Cassandra schema follows a non-relational, high-performance design optimized for high-throughput read/write operations and scalability. Data is organized into column families (analogous to tables in relational databases), with a primary key uniquely identifying each row. For example, a column family may store basic product information using the product ID as the primary key. Additional attributes such as seller or variant details may be stored in the same or separate column families, depending on query requirements.

Defining schemas for both PostgreSQL and Cassandra enables the system to leverage the strengths of each database. PostgreSQL supports complex queries, analytical reporting, and high data consistency, while Cassandra supports fast access and scalable storage of frequently accessed product data.

After receiving JSON responses from Lazada’s API, data is parsed using Python’s `json` library. The parsed data is mapped to the schema structures of PostgreSQL and Cassandra. This mapping includes data cleaning, type conversion, and handling of missing or invalid values. The processed data is then inserted into the respective databases.

During data collection, attention is paid to Lazada’s API rate limits to avoid service disruption. The application is designed to comply with these limits—for example, by spacing out API calls using sleep intervals if necessary. Robust error handling is also implemented to manage issues such as network failures, authentication errors, or server-side problems from Lazada.

Attribute	Data Type	Description
item_id	Int	Unique identifier of the product
name	text	Name of the product
created_time	Date.now()	Product creation timestamp
updated_time	Date.now()	Most recent update timestamp
primary_category	text	Main category of the product
status	boolean	Product status
created_date	Date.now()	Record creation timestamp

Table 4.1: Description of product information

Attribute	Data Type	Description
item_id	Int	Foreign key linking to the products table
sku_id	Int	Unique identifier for the SKU
seller_sku	NVARCHAR	SKU code assigned by the seller
shop_sku	NVARCHAR	SKU code assigned by the shop
status	boolean	Status of the SKU
created_date	Date.now()	Record creation timestamp

Table 4.2: Description of SKU information

The two tables presented earlier describe the fundamental structure of product-related

data on the Lazada platform. Among all attributes, the most critical one is `item_id`, as it uniquely identifies each product and serves as a foreign key linking to the SKUs table. Other attributes, including product name, category, status, creation time, and update time, are also essential as they provide users with a comprehensive view of each product. Notably, the SKUs table contains detailed information regarding product variants, such as pricing, inventory levels, and business status.

The ingestion process involves two complementary data entry paths:

1. **Realtime API Ingestion (Primary Flow):** The system uses Python's `requests` library to fetch live product and SKU data from Lazada's REST API. After authentication and request execution, the raw JSON responses are cleaned, validated, and serialized into Kafka topics such as `lazada-products` and `lazada-skus`. This flow is designed for near real-time ingestion of active product listings.
2. **Extract: Lazada API to PostgreSQL Staging:** As part of the Extract phase in the ETL pipeline, the system incorporates a PostgreSQL-based staging layer to temporarily store structured product data retrieved from the Lazada Open Platform API. This staging mechanism plays a crucial role in decoupling the data acquisition process from downstream processing, enabling robust validation, schema enforcement, and error handling before data is forwarded to Kafka.

The extraction workflow is implemented using Python. Upon execution, the script establishes a connection to the PostgreSQL database with connection parameters loaded securely from environment variables. It then ensures that all required tables are created with appropriate primary keys and foreign key constraints to maintain referential integrity.

Product data is fetched from the Lazada API through paginated HTTP requests. Each API response is parsed and validated before being inserted into the corresponding PostgreSQL tables. To ensure idempotency and support incremental extraction, the system leverages PostgreSQL's `ON CONFLICT` clause to perform upserts—automatically updating existing records or inserting new entries as appropriate. Additionally, all records are tagged with a `created_date` field to facilitate version tracking and time-based filtering.

By introducing this staging layer, the system gains enhanced control over data ingestion, making it easier to audit, debug, and recover from partial failures. It also enables batch-level inspection of collected data before it enters the Kafka pipeline, ensuring that only clean, validated, and well-structured data proceeds to the trans-

formation and loading phases.

```
def process_and_save_data(run_migration=False):
    conn = connect_to_db()
    if run_migration:
        create_tables(conn)

    page = 1
    today = datetime.now().date()
    while True:
        data = fetch_tiki_data(page)
        if not data or not data.get('data', {}).get('products'):
            break
        products = data['data']['products']

        with conn.cursor() as cur:
            for product in products:
                cur.execute("""
                    INSERT INTO lazada_products (item_id, name,
                        created_time, updated_time,
                        primary_category, status, created_date)
                    VALUES (%s, %s, to_timestamp(%s::bigint/1000),
                        to_timestamp(%s::bigint/1000),
                        %s, %s, %s)
                    ON CONFLICT (item_id) DO UPDATE SET
                        name = EXCLUDED.name,
                        updated_time = EXCLUDED.updated_time
                """, (
                    product.get('item_id'),
                    product['attributes'].get('name'),
                    product.get('created_time'),
                    product.get('updated_time'),
                    product.get('primary_category'),
                    product.get('status'),
                    today
                ))
            conn.commit()
        page += 1
```

Listing 4.1: Extracting product data from Lazada API and storing to PostgreSQL

4.3.2 Batch Layer

This is the batch layer in the Lambda architecture, where data is persistently stored in HDFS. The data is saved in Parquet format to optimize query performance and reduce

storage space.

```
def save_to_hdfs(df, epoch_id):
    try:
        hdfs_path = f"hdfs://namenode:9000/data/postgres/lazada/
                     batch_{epoch_id}"
        df.write \
            .mode("append") \
            .parquet(hdfs_path)
        logger.info(f"Successfully saved batch {epoch_id} to HDFS")
    except Exception as e:
        logger.error(f"Error saving to HDFS: {e}")

def process_stream():
    try:
        spark = create_spark_session()
        hdfs_paths = [
            "hdfs://namenode:9000/data",
            "hdfs://namenode:9000/data/postgres",
            "hdfs://namenode:9000/data/postgres/lazada"
        ]
        for path in hdfs_paths:
            spark._jvm.org.apache.hadoop.fs.FileSystem.get(
                spark._jsc.hadoopConfiguration()) \
                .mkdirs(spark._jvm.org.apache.hadoop.fs.Path(path))

        df = spark.readStream \
            .format("kafka") \
            .option("kafka.bootstrap.servers", "localhost:9092") \
            .option("subscribe", "lazada-products") \
            .load()

        query = df.writeStream \
            .foreachBatch(save_to_hdfs) \
            .option("checkpointLocation", "hdfs://namenode:9000/
                     checkpoints") \
            .start()
```

Listing 4.2: Spark Structured Streaming to HDFS

The second task, `save_to_nifi`, is responsible for sending processed data batches to Apache NiFi for downstream dataflow management. It converts each record from the DataFrame into JSON format and transmits them to NiFi's REST API endpoint. The function establishes a connection to a running NiFi instance on a configured host and port, then sends each record as an individual HTTP request to the appropriate processor group. This integration allows the ETL pipeline to leverage NiFi's powerful capabilities in data routing, transformation, and system orchestration for further processing.

```
def save_to_nifi(batch_df, batch_id):
    try:
        json_data = batch_df.toJSON().collect()

        nifi_url = f"http://{getenv('NIFI_HOST')}:8080/nifi-api/
                    process-groups/root/processors"
        headers = {
            'Content-Type': 'application/json'
        }

        for record in json_data:
            response = requests.post(nifi_url, headers=headers, data=
                                    record)
            response.raise_for_status()

        logger.info(f"Batch {batch_id} sent to NiFi")
    except Exception as e:
        logger.error(f"Failed to send batch {batch_id} to NiFi: {str(
            e)}")
```

Listing 4.3: Task to send data to Apache NiFi

The `db_to_kafka.py` script acts as a bridge to publish cleaned data already stored in the PostgreSQL database into Kafka topics. Although the main ingestion flow interacts directly with the Lazada API, this batch re-ingestion script is useful in the following scenarios:

- Replaying historical data or recovering failed records.
- Populating Kafka topics for local development and testing without making repeated API calls.
- Bootstrapping the pipeline with initial data from PostgreSQL before live API sync begins.

By aligning with the same Kafka topic schema, this script ensures both real-time and

batch flows are compatible and interoperable.

```
import psycopg2
from kafka import KafkaProducer
from kafka.admin import KafkaAdminClient, NewTopic
import json
from datetime import datetime, date
from decimal import Decimal
import logging

# Setup logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(
    levelname)s - %(message)s')
logger = logging.getLogger(__name__)

# PostgreSQL configuration
DB_CONFIG = {
    'host': 'localhost',
    'port': 5433,
    'database': 'tiki_etl',
    'user': 'postgres2',
    'password': 'abc12345'
}

# Kafka configuration
KAFKA_BOOTSTRAP_SERVERS = 'localhost:9092'

# Safe value formatter for JSON serialization
def safe_value(v):
    try:
        if isinstance(v, (datetime, date)):
            return v.isoformat()
        elif isinstance(v, Decimal):
            return float(v)
        elif isinstance(v, bytes):
            return v.decode('utf-8')
        elif isinstance(v, str):
            if v.isdigit():
                return int(v)
            try:
                return float(v)
            except:
                return v
        elif isinstance(v, (int, float, bool, type(None))):
```

```

        return v
    else:
        return str(v)
except:
    return None

# Connect to PostgreSQL
def connect_to_db():
    try:
        conn = psycopg2.connect(**DB_CONFIG)
        logger.info("    Connected to PostgreSQL successfully.")
        return conn
    except Exception as e:
        logger.error(f"    Database connection error: {e}")
        raise

# Create Kafka producer
def create_kafka_producer():
    return KafkaProducer(
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        value_serializer=lambda v: json.dumps(v, default=safe_value).
            encode('utf-8')
    )

# Convert SQL result rows to list of dictionaries
def rows_to_dict(cursor, rows):
    columns = [desc[0] for desc in cursor.description]
    return [dict(zip(columns, row)) for row in rows]

# Ensure topic exists in Kafka, create if missing
def ensure_topic_exists(topic_name):
    try:
        admin_client = KafkaAdminClient(
            bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
            client_id='kafka-admin'
        )
        topic_list = admin_client.list_topics()
        if topic_name not in topic_list:
            topic = NewTopic(name=topic_name, num_partitions=1,
                replication_factor=1)
            admin_client.create_topics([topic])
            logger.info(f"    i    Created topic: {topic_name}")
        else:

```

```
        logger.info(f"                Topic already exists: {topic_name}")
    admin_client.close()
except Exception as e:
    logger.warning(f"                Unable to verify/create topic {
        topic_name}: {e}")

# Main function to fetch from DB and send to Kafka
def fetch_and_send():
    conn = None
    producer = None
    try:
        conn = connect_to_db()
        producer = create_kafka_producer()
        cursor = conn.cursor()

        tables = {
            'lazada-products': "SELECT * FROM lazada_products;",
            'lazada-skus': "SELECT * FROM lazada_skus;",
            # 'lazada-images': "SELECT * FROM lazada_images;",
            # 'lazada-attributes': "SELECT * FROM lazada_attributes;"
        }

        for topic, query in tables.items():
            ensure_topic_exists(topic)
            cursor.execute(query)
            rows = cursor.fetchall()
            records = rows_to_dict(cursor, rows)

            for record in records:
                safe_record = {k: safe_value(v) for k, v in record.
                    items()}
                producer.send(topic, value=safe_record)
                logger.info(f" Sent to {topic}: {safe_record}")

        producer.flush()
        logger.info(" All data sent successfully to Kafka.")

    except Exception as e:
        logger.error(f" Error during data fetch/send: {e}")
    finally:
        if producer:
            producer.close()
        if conn:
```

```
conn.close()

if __name__ == "__main__":
    fetch_and_send()
```

Upon execution, the task establishes a secure connection to a local PostgreSQL database using the `psycopg2` library. It retrieves all rows from relevant tables which contain information about product listings and stock keeping units (SKUs). The data is extracted using standard SQL queries and converted into Python dictionaries for structured handling.

To facilitate data serialization, the script defines a custom `safe_value()` function. This function safely converts complex data types—such as `datetime`, `Decimal`, and `bytes`—into JSON-compatible formats. This ensures that all values are correctly formatted before being sent to Kafka.

Before transmitting data, the script utilizes `KafkaAdminClient` from the `kafka-python` library to verify the existence of Kafka topics such as `"lazada-products"` and `"lazada-skus"`. If a topic does not exist, it is created dynamically with a single partition and a replication factor of one. This design allows the ingestion process to run smoothly without requiring prior manual topic configuration.

Once the topics are confirmed, the script initializes a `KafkaProducer` and begins publishing data. Each record is serialized to JSON and sent individually to the corresponding topic. During the process, detailed logs are recorded to provide full traceability of each message that is published.

Finally, the producer is flushed to ensure all messages are delivered, and all connections are properly closed to release system resources. In case of any error—whether from database connection, Kafka interaction, or data formatting—the script captures and logs the exception.

In summary, this task automates the end-to-end process of transferring structured product and inventory data from PostgreSQL into Kafka. By incorporating topic creation and robust serialization, it lays a reliable foundation for real-time analytics, stream processing, and monitoring in the subsequent stages of the data pipeline.

4.3.3 Data Processing and Storage Layer

a, Database Design

The database design process is a critical step in constructing a robust data storage system. A well-designed database enables organized and efficient data management, optimizes query performance, and ensures data integrity. In this section, I present the database design for the storage system implemented using Apache Cassandra within this layer.

Attribute	Data Type	Description
id	Big_int	Unique identifier for the product
name	text	Product name
created_time	Date.now()	Timestamp when the product was listed
updated_time	Date.now()	Timestamp when the product was last updated
primary_category	Object	Product's main category
created_date	Date.now()	Record creation time

Table 4.3: Basic product information

Attribute	Data Type	Description
item_id	Big_int	Composite key with sku_id
sku_id	Big_int	Unique identifier of the SKU
seller_sku	Text	SKU code defined by the seller
shop_sku	Text	SKU code specific to the shop
quantity	Int	Stock quantity
available	Int	Quantity available for sale
price	Int	Original price
special_price	Int	Promotional price

Table 4.4: Variant-level information

The following formulas are applied to the data presented in the SKU table:

- **Displayed Price** = $\min(\text{price}, \text{special_price})$ when $\text{special_price} > 0$
- **Discount Percentage** = $\frac{\text{price} - \text{special_price}}{\text{price}} \times 100\%$
- **Discounted Price** = $\text{price} - (\text{price} \times \text{discount_percentage})$
- **Profit** = $\text{selling_price} - (\text{cost_price} + \text{overhead_costs})$
- **ROI (Return on Investment)** = $\frac{\text{profit}}{\text{cost_price}} \times 100\%$

The table also tracks inventory, where `quantity` represents the total number of items in stock, and `available` indicates the number of items currently available for sale.

Attribute	Data Type	Description
<code>item_id</code>	<code>Big_int</code>	Composite key with <code>image_url</code>
<code>image_url</code>	<code>Text</code>	Image URL of the product
<code>created_date</code>	<code>Date.now()</code>	Record creation date

Table 4.5: Product image metadata

Table 5 is designed to store multiple images per product using the `item_id` as a clustering key to optimize image gallery retrieval.

Attribute	Data Type	Description
<code>item_id</code>	<code>Big_int</code>	Primary key
<code>brand</code>	<code>Text</code>	Brand name
<code>description</code>	<code>Text</code>	Product description
<code>created_date</code>	<code>Date.now()</code>	Record creation date

Table 4.6: Brand and product description

Table 6 contains extended information such as brand and detailed product descriptions. These are separated to avoid heavy loads when retrieving only the basic product information.

With this, the design for the data storage layer is complete. The next section will focus on the two key components of this layer: the data processing logic and data visualization modules.

b, Processing and Storage Workflow

The data processing and storage workflow begins by initializing connections to both Apache Spark and Apache Cassandra. A `SparkSession` is configured with the necessary parameters to handle streaming data from the Kafka topic "`lazada-products`" and to interact with Cassandra. The Cassandra connection includes authentication and is set up to automatically create the required keyspace if it does not already exist.

Spark is responsible for ingesting streaming data from Kafka, applying necessary transformations, and persisting the cleaned and structured data to Cassandra. The Cassandra schema is designed to efficiently store product-related information and consists of four main tables: `products`, `skus`, `images`, and `attributes`.

The first step in this workflow involves creating the `SparkSession` with appropriate configurations to establish the connection with the Cassandra cluster.

```
def create_spark_session():
    os.environ['JAVA_HOME'] = r'C:\Program Files\Java\jdk1.8.0_211'
    os.environ['HADOOP_HOME'] = r'C:\Hadoop\hadoop-3.3.6'
    os.environ['PATH'] = f"{os.environ['HADOOP_HOME']}\bin;{os.
        environ['PATH']}"
    print("==> STEP 1: Creating SparkSession...")
    spark = SparkSession.builder \
        .appName("TikiConsumer") \
        .master("local[*]") \
        .config("spark.jars.packages", "org.apache.spark:spark-sql-
            kafka-0-10_2.12:3.0.3") \
        .config("spark.ui.port", "4040") \
        .config("spark.driver.host", "127.0.0.1") \
        .config("spark.driver.bindAddress", "127.0.0.1") \
        .config("spark.hadoop.fs.hdfs.impl", "org.apache.hadoop.hdfs.
            DistributedFileSystem") \
        .config("spark.metrics.namespace", "spark") \
        .getOrCreate()
    print("==> STEP 2: SparkSession created successfully.")
    spark.sparkContext.setLogLevel("WARN")
    return spark
```

Listing 4.4: Function to initialize `SparkSession`

```
def create_cassandra_connection():
    try:
        cluster = Cluster(
            [getenv('CASSANDRA_HOST', 'localhost')],
            port=int(getenv('CASSANDRA_PORT', 9042)),
            auth_provider=PlainTextAuthProvider(
                username=getenv('CASSANDRA_USER', 'cassandra'),
                password=getenv('CASSANDRA_PASSWORD', 'cassandra')
            )
        )
        session = cluster.connect()
        logger.info("Cassandra connection established successfully")

        # Create keyspace if not exists
        session.execute("""
            CREATE KEYSPACE IF NOT EXISTS lazada_store
            WITH replication = {
                'class': 'SimpleStrategy',
                'replication_factor': 1
            }
        """)
        session.set_keyspace('lazada_store')

        return session
    except Exception as e:
        logger.error(f"Failed to create Cassandra connection: {e}")
    return None
```

Listing 4.5: Function to establish connection with Cassandra and create keyspace

The process begins by defining the schema for Lazada product data, aligned with the table structures in Cassandra. The system then establishes a streaming connection with Kafka to continuously receive product data in real time.

```
lazada_schema = StructType([
    StructField("product", StructType([
        StructField("item_id", LongType()),
        StructField("name", StringType()),
        StructField("created_time", TimestampType()),
        StructField("updated_time", TimestampType()),
        StructField("primary_category", IntegerType()),
        StructField("status", StringType())
    ])),

    StructField("skus", ArrayType(StructType([
        StructField("item_id", LongType()),
        StructField("sku_id", LongType()),
        StructField("seller_sku", StringType()),
        StructField("shop_sku", StringType()),
        StructField("status", StringType()),
        StructField("quantity", IntegerType()),
        StructField("available", IntegerType()),
        StructField("price", DoubleType()),
        StructField("special_price", DoubleType())
    ]))),

    StructField("images", ArrayType(StructType([
        StructField("item_id", LongType()),
        StructField("image_url", StringType())
    ]))),

    StructField("attributes", StructType([
        StructField("item_id", LongType()),
        StructField("brand", StringType()),
        StructField("description", StringType())
    ]))
])
```

Listing 4.6: Schema definition for Lazada product data

Subsequently, a streaming connection is established with Kafka to continuously ingest Lazada product data using the previously defined schema. This schema encompasses core product information, SKU data, images, and additional product attributes, ensuring structured and efficient data processing.

```
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", f"{getenv('KAFKA_SERVERS', 'localhost:9092')}") \
    .option("subscribe", "lazada-products") \
    .option("startingOffsets", "earliest") \
    .option("failOnDataLoss", "false") \
    .load()

parsed_df = df.select(
    from_json(col("value").cast("string"), lazada_schema).alias("data")
).select(
    col("data.product.item_id").alias("item_id"),
    col("data.product.name").alias("name"),
    col("data.product.created_time").alias("created_time"),
    col("data.product.updated_time").alias("updated_time"),
    col("data.product.primary_category").alias("primary_category"),
    col("data.product.status").alias("status"),
    col("data.skus")[0]["price"].alias("price"),
    col("data.skus")[0]["quantity"].alias("quantity"),
    col("data.images")[0]["image_url"].alias("image_url"),
    col("data.attributes.brand").alias("brand"),
    col("data.attributes.description").alias("description"),
    current_timestamp().alias("processing_time")
)
```

Listing 4.7: Reading Lazada data from Kafka and parsing it using Spark Structured Streaming

After receiving data from Kafka, the system separates it into distinct data streams for each type of information. Each of these streams is processed independently.

```
# Save to HDFS
query1 = parsed_df.writeStream \
    .foreachBatch(save_to_hdfs) \
    .start()
```

Listing 4.8: Write processed data to HDFS

```
# Save to Cassandra
query2 = parsed_df.writeStream \
    .foreachBatch(save_to_cassandra) \
    .start()
```

Listing 4.9: Write processed data to Cassandra

```
# Save to PostgreSQL
query3 = parsed_df.writeStream \
    .foreachBatch(save_to_postgres) \
    .start()
```

Listing 4.10: Write processed data to PostgreSQL

After receiving data from Kafka, the system separates it into distinct data streams for each type of information, such as product details, SKUs, images, and attributes. Each stream is processed independently to ensure modularity and scalability. To enhance flexibility and performance, a multisink architecture is employed, allowing the same stream to be routed to multiple storage systems simultaneously—such as PostgreSQL for relational queries, Cassandra for high-throughput access, and HDFS for long-term storage. This design simplifies system expansion and enables downstream components (e.g., analytics dashboards or alerting services) to consume data from the most appropriate source, thus optimizing performance and scalability.

4.3.4 Data Serving and Visualization Layer

As previously presented in Chapter 3, I utilized Prometheus, Grafana, and Superset to implement the monitoring and data visualization system. This monitoring framework plays a crucial role in tracking and visually displaying system metrics related to the Lazada data pipeline.

Prometheus is responsible for collecting and storing metrics from various sources, including PostgreSQL, Cassandra, and other system services. Grafana then provides an interactive dashboard interface that allows users to monitor and analyze the following aspects:

1. **Product Price Trends:** Visualization of historical and current price fluctuations.
2. **Technical Indicators:** Monitoring of key metrics and computed trends to identify anomalies and significant changes over time.
3. **System Status and Service Performance:** Real-time insight into the health and efficiency of various services.

4. **Alerts on Price or Inventory Anomalies:** Detecting and notifying abnormal fluctuations in product pricing or stock levels.

The system is configured with custom alert rules to detect anomalies and promptly notify administrators. Grafana dashboards are designed for ease of use, enabling users to monitor and analyze data trends in real time.

```
# price-rules
groups:
- name: lazada_price_changes
  rules:
    - alert: PriceVolatility
      expr: changes(lazada_product_price[1h]) > 5
      for: 15m
      labels:
        severity: warning
      annotations:
        summary: "High price volatility detected"
        description: "Product {{ $labels.name }} price changed more
          than 5 times in last hour"

    - alert: TechnicalIndicatorAlert
      expr: lazada_price_rsi > 70 or lazada_price_rsi < 30
      for: 30m
      labels:
        severity: info
      annotations:
        summary: "Technical indicator alert"
        description: "RSI for product {{ $labels.name }} is in
          overbought/oversold territory"

# product-alerts
groups:
- name: lazada_product_alerts
  rules:
    - alert: ProductPriceSpike
      expr: rate(lazada_product_price[5m]) > 0.2
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Product price increased significantly"
        description: "Price for product {{ $labels.name }} has
          increased by more than 20% in 5 minutes"

    - alert: LowInventory
      expr: lazada_product_quantity < 10
      for: 10m
      labels:
```

```

severity: warning
annotations:
  summary: "Low product inventory"
  description: "Product {{ $labels.name }} has less than 10
               items in stock"

```

Listing 4.11: Prometheus Alert Rules

```

global:
  scrape_interval: 15s
  evaluation_interval: 15s
  scrape_timeout: 10s

scrape_configs:
  - job_name: 'lazada_metrics'
    static_configs:
      - targets: ['localhost:8000']
    metrics_path: '/metrics'
    scheme: 'http'

  - job_name: 'kafka'
    static_configs:
      - targets: ['kafka-exporter:9308']

  - job_name: 'lazada_product_metrics'
    static_configs:
      - targets: ['localhost:8001']
    metrics_path: '/product-metrics'
    scrape_interval: 30s

  - job_name: 'cassandra_metrics'
    static_configs:
      - targets: ['localhost:9042']
    metrics_path: '/metrics'
    scrape_interval: 1m

  - job_name: 'spark'
    metrics_path: '/metrics/prometheus'
    static_configs:
      - targets: ['host.docker.internal:4040']

rule_files:
  - 'rules/product_alerts.yml'

```

```
- 'rules/price_changes.yml'

alerting:
  alertmanagers:
    - static_configs:
      - targets: ['localhost:9093']
```

Listing 4.12: Prometheus Configuration File

The data processing and monitoring workflow in the system is designed to provide real-time observability across the entire data pipeline. Prometheus acts as the central metrics collector, configured to scrape data from multiple endpoints including:

- *localhost:8000* and *localhost:8001*: Intended to expose application-level business metrics such as product prices, inventory levels, and promotional activity. However, the current Prometheus status indicates these endpoints are unreachable, suggesting that the services or exporters may not be running.
- *host.docker.internal:4040/metrics/prometheus*: Serves Apache Spark job metrics, including garbage collection (GC) time, number of GC events, and active streaming job status.
- *kafka-exporter:9308*: Provides Kafka metrics such as per-topic message counts, partition details, consumer group offsets, and total message lag.
- *localhost:9042*: Configured as the source for Cassandra metrics, though the default port only returns basic Prometheus scrape duration. For full observability, a proper Cassandra HTTP metrics exporter is required.

The collected metrics are organized into five logical groups, each defined in the Prometheus configuration via *job_name* and *scrape_interval* parameters:

- **Basic Lazada Metrics** (*job_name: lazada_metrics, scrape_interval: 15s*):
Intended to monitor general product metrics such as price, quantity, and status from *localhost:8000*.
- **Detailed Product Metrics** (*job_name: lazada_product_metrics, scrape_interval: 30s*):
Exposed via *localhost:8001*, designed for detailed analyses including price volatility, statistical trends, and category aggregations.
- **Kafka Exporter Metrics** (*job_name: kafka, scrape_interval: 15s*):
Collected from *kafka-exporter:9308*, used to monitor Kafka broker health, topic

throughput, consumer lag, and offset progression.

- **Apache Spark Metrics** (*job_name: spark*):
Accessed via *host.docker.internal:4040*, covers Spark executor status, streaming rates, garbage collection, and job execution metrics.
- **Cassandra Metrics** (*job_name: cassandra_metrics, scrape_interval: 1m*):
Aimed at tracking database performance such as read/write latency and compaction, though currently only basic scrape duration is recorded.

Prometheus stores all metrics as time-series data and continuously evaluates alerting rules defined in *rules/product_alerts.yml* and *rules/price_changes.yml*. These rules include:

- *PriceVolatility* and *TechnicalIndicatorAlert* for detecting statistical anomalies in price movement.
- *ProductPriceSpike* and *LowInventory* for identifying critical business conditions.

Alerts are sent to administrators via Alertmanager (*localhost:9093*) whenever predefined thresholds are breached, allowing fast response to issues.

All collected data is visualized through Grafana dashboards. These dashboards provide:

- Real-time Kafka topic monitoring, including partition counts and message volumes exceeding 15 million records per topic.
- Spark GC time and job activity graphs for tracking ETL stability.
- Cassandra scrape latency tracking, pending full integration with a dedicated exporter.
- Live feedback on alert status, evaluation frequency, and execution latency.

Together, this monitoring stack offers robust observability for both infrastructure and business layers. It enables system administrators and analysts to track trends, diagnose issues, and make data-driven operational decisions in real time across the Lazada analytics ecosystem.

CHAPTER 5. SYSTEM DEPLOYMENT AND EXPERIMENTAL RESULTS

In the previous chapter, I presented a detailed description of the system architecture. In this chapter, I will first describe the deployment of the system using Docker containers. Then, I will present the experimental results obtained after deployment, followed by an evaluation of the system through testing.

5.1 System Deployment

As discussed in Chapter 3, Docker is an open-source platform that enables the development, deployment, and management of applications within containerized environments. By leveraging Docker, the system can be efficiently deployed and orchestrated across multiple services on a single host machine. This approach simplifies configuration, enhances portability, and improves scalability.

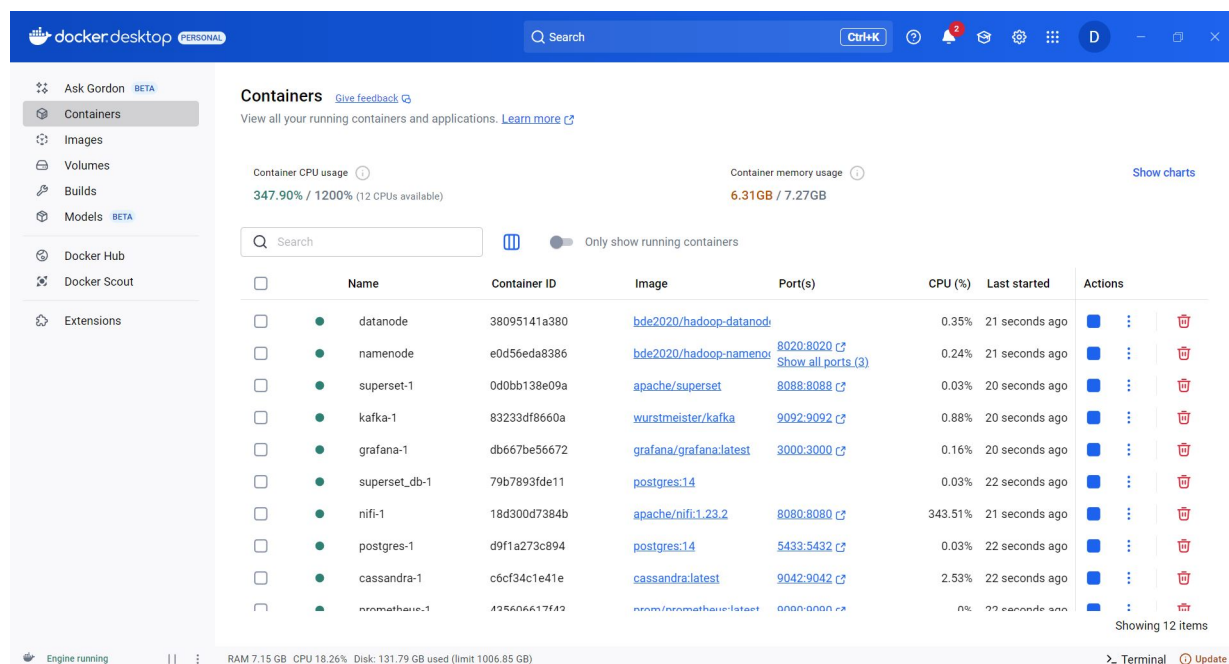


Figure 5.1: Information about containers

Figure 5.1 illustrates an overview of the deployed containers. Below is a detailed description of the system deployment environment and container configurations.

5.1.1 Deployment Environment

- **Operating System:** Windows 11 23H2
- **Memory:** 16 GB
- **Processor:** AMD Ryzen 5 R5-5800H (6 cores, 12 threads)

- **Application Platform:** Docker Desktop 4.31.1

5.1.2 Deployed Containers

a, Monitoring and Visualization System

Prometheus (port 9090)

- Image: `prom/prometheus:latest`
- Role: Collect and store metrics
- Main configuration:
 - Config file: `/etc/prometheus/prometheus.yml`
 - Rules directory: `/etc/prometheus/rules`
 - Storage path: `/prometheus`
- Volume: `prometheus_data` for time-series data
- Network: `app_network`

Grafana (port 3000)

- Image: `grafana/grafana:latest`
- Role: Visualization and alerting
- Configuration:
 - Admin credentials: `admin/admin`
 - Sign-up disabled for security
 - Provisioned dashboards and datasources
- Volume: `grafana_data` for persistent storage
- Dependency: Prometheus
- Network: `app_network`

b, Message Queue System

Zookeeper (port 2181)

- Image: `wurstmeister/zookeeper`
- Role: Kafka cluster manager

Kafka (port 9092)

- Image: `wurstmeister/kafka`

- Role: Message broker
- Configuration:
 - Advertised host: `localhost`
 - Zookeeper connection: `zookeeper:2181`
 - Auto-create topics: `"tiki-products:1:1"`
- Dependency: Zookeeper

Kafka Exporter (port 9308)

- Image: `danielqsj/kafka-exporter`
- Role: Kafka metrics exporter for Prometheus
- Configuration:
 - Kafka server: `kafka:29092`
 - Kafka version: `2.0.0`
 - Exposed metrics endpoint: `:9308`
- Dependency: Kafka

c, Storage System

Hadoop

NameNode (ports 9870, 9000, 8020)

- Image: `bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8`
- Role: Metadata manager and cluster coordinator
- Configuration: `HDFS replication = 1, defaultFS = hdfs://namenode:9000`
- Volumes: `hadoop_namenode, hadoop_data`
- Network: `app_network`

DataNode

- Image: `bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8`
- Role: Distributed data storage
- Configuration: `Connect to NameNode`
- Volume: `hadoop_datanode`
- Dependency: `NameNode`

- Network: `app_network`

PostgreSQL (port 5433)

- Image: `postgres:14`
- Role: Relational database
- Configuration:
 - Database: `tiki_etl`
 - User: `postgres2`
 - Password: `abc12345`
- Volume: `postgres_data`
- Network: `app_network`

Cassandra (port 9042)

- Image: `cassandra:latest`
- Role: NoSQL time-series database

d, Data Ingestion and Processing

Apache Superset (port 8088)

- Image: `apache/superset`
- Role: Data exploration and visualization
- Configuration:
 - Secret key setup
 - Database connection via environment variables
- Volume: `./superset` for `superset_home`
- Dependency: PostgreSQL
- Network: `app_network`

Superset Database

- Image: `postgres:14`
- Role: Metadata storage for Superset
- Configuration:
 - Database: `superset`

- User: superset
- Password: superset123
- Volume: superset_db_data

5.1.3 Network Configuration

- Network: Bridge network `app_network` for internal container communication
- Exposed ports for external access from host machine

5.1.4 Volume Management

Persistent volumes for critical data:

- `hadoop_namenode`, `hadoop_datanode`
- `postgres_data`
- `prometheus_data`
- `grafana_data`
- `superset_db_data`

5.1.5 Deployment Features

- Profile `infra` used for all services to simplify deployment
- Explicitly defined dependencies among services
- Volume persistence to prevent data loss on container restart

The system is architected based on a Lambda approach, integrating both batch and streaming data processing along with modern tools for visualization and monitoring. The use of Docker containers facilitates straightforward deployment and scalability.

5.2 Experimental Results

5.2.1 Data Collection Experiment

To evaluate the effectiveness of the data collection process and ensure that the monitoring system for the Lazada data pipeline operates as expected, the following verification steps were carried out:

1. Prometheus Interface Inspection (<http://localhost:9090>):

- Accessed the *Targets* tab to verify that all exporters were properly connected and pushing metrics.
- Queried key metrics such as `up`, `lazada_scraper_runtime_seconds`, and `kafka_topic_partition_current_offset` in the *Graph* tab to

ensure real-time metric collection.

- Reviewed and confirmed the alerting rules were correctly defined and functioning as intended.

2. Grafana Dashboard Monitoring (<http://localhost:3000>):

- Logged in using the default credentials: `admin/admin`.
- Verified that all required dashboards were successfully imported.
- Checked real-time data visualization and confirmed metrics were rendered correctly on various panels.
- Validated the status and logic of the configured alert conditions.

3. Apache NiFi Flow Monitoring (<http://localhost:8080>):

- Logged in with the credentials: `admin/ctmaicungduoc`.
- Verified that all data processors were active and functioning within the designed flow.
- Inspected the status of flowfiles in transit to ensure proper data processing.
- Reviewed processor-specific logs to debug and resolve issues, if any.

4. PostgreSQL Database Validation (Port 5433):

- Connected to the `tiki_etl` database to verify the presence of product data.
- Executed SQL queries to ensure data consistency and completeness.
- Confirmed that historical metrics were properly stored in the respective tables.

5. Apache Spark Structured Streaming Monitoring (<http://localhost:4040>):

- Accessed Spark UI via port 4040 to monitor running jobs and batch execution status.
- Confirmed that Spark jobs triggered by the ETL process (e.g., `tiki_consumer.py`) were properly initialized and maintained during execution.
- Verified job metadata including execution time, job ID, and task progress through the Spark Job Timeline.

These verification steps collectively ensured that the entire pipeline—from metric collection, data processing, storage, to visualization—was operating in accordance with the system design. In the case of any discrepancies or failures, the logs from individual services were utilized for effective debugging and resolution.

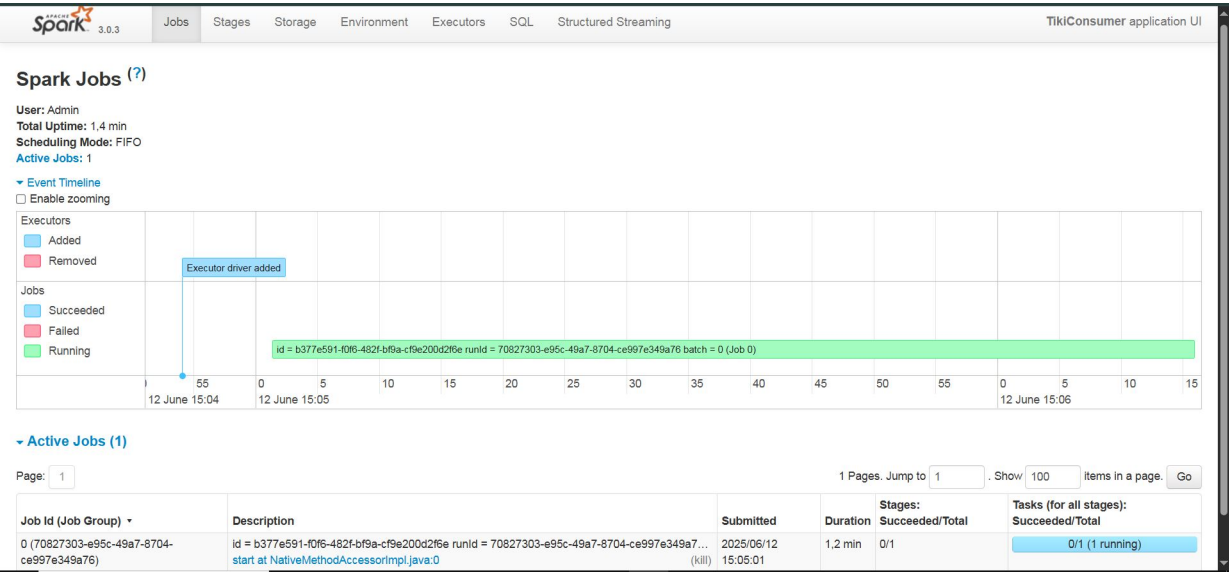


Figure 5.2: Spark job tracking interface

Figure 5.2 illustrates the real-time Spark job tracking interface confirming that the system was actively streaming and processing data.

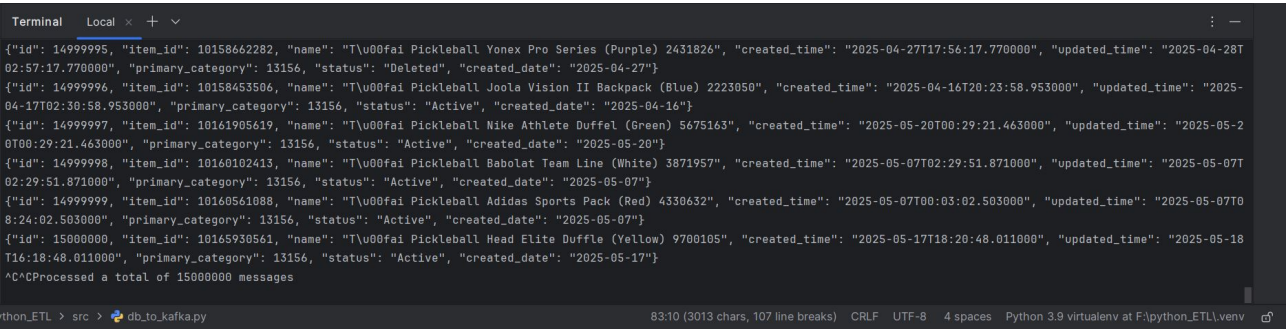


Figure 5.3: Data Collection into Kafka

The logs shown in Figure 5.3 confirm that the data was successfully collected and fully transmitted to the Kafka topic without any loss or interruption.

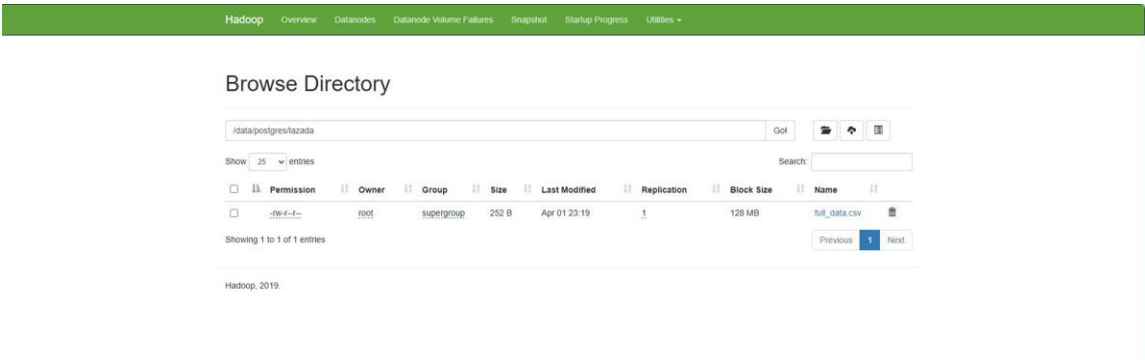


Figure 5.4: Data Collection into Hadoop

The data retrieved from the PostgreSQL database was fully collected and successfully transmitted to the Hadoop storage system.

5.2.2 Data Processing and Storage Experiment

Regarding the experimental results of data processing and storage, I verified whether the data was fully stored in the pre-designed tables of both Cassandra and PostgreSQL. This involved checking the structure, consistency, and completeness of the inserted records to ensure the data pipeline operated as intended.

item_id	name	created_time	updated_time	primary_category	status	created
3056230414	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05
4056230415	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05
4056230416	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05
4056230417	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05
4056230418	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05
4056230419	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05
4056230420	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05
4056230421	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05
4056230422	Túi Pickleball Joola Vision II Backpack (BL...	2025-03-24T16:50:04	2025-03-24T16:50:04	13156	Active	2025-05

Figure 5.5: Data Processing and Storage

id	item_id	sku_id	seller_sku	shop_sku	status	quantity	value
1	6056230415	44701183682	12345KU_1	6056230415_VNAMZ-44701183682	active	100	
2	6056230416	44701183684	12345KU_2	6056230416_VNAMZ-44701183684	active	100	
3	6056230417	44701183685	12345KU_3	6056230417_VNAMZ-44701183685	active	100	
4	6056230418	44701183686	12345KU_4	6056230418_VNAMZ-44701183686	active	100	
5	6056230419	44701183687	12345KU_5	6056230419_VNAMZ-44701183687	active	100	
6	6056230420	44701183688	12345KU_6	6056230420_VNAMZ-44701183688	active	100	
7	6056230421	44701183689	12345KU_7	6056230421_VNAMZ-44701183689	active	100	
8	6056230422	44701183690	12345KU_8	6056230422_VNAMZ-44701183690	active	100	
9	6056230423	44701183691	12345KU_9	6056230423_VNAMZ-44701183691	active	100	
10	6056230424	44701183692	12345KU_10	6056230424_VNAMZ-44701183692	active	100	
11	6056230425	44701183693	12345KU_11	6056230425_VNAMZ-44701183693	active	100	
12	6056230426	44701183694	12345KU_12	6056230426_VNAMZ-44701183694	active	100	
13	6056230427	44701183695	12345KU_13	6056230427_VNAMZ-44701183695	active	100	
14	6056230428	44701183696	12345KU_14	6056230428_VNAMZ-44701183696	active	100	
15	6056230429	44701183697	12345KU_15	6056230429_VNAMZ-44701183697	active	100	
16	6056230430	44701183698	12345KU_16	6056230430_VNAMZ-44701183698	active	100	
17	6056230431	44701183699	12345KU_17	6056230431_VNAMZ-44701183699	active	100	
18	6056230432	44701183700	12345KU_18	6056230432_VNAMZ-44701183700	active	100	
19	6056230433	44701183701	12345KU_19	6056230433_VNAMZ-44701183701	active	100	
20	6056230434	44701183702	12345KU_20	6056230434_VNAMZ-44701183702	active	100	
21	6056230435	44701183703	12345KU_21	6056230435_VNAMZ-44701183703	active	100	
22	6056230436	44701183704	12345KU_22	6056230436_VNAMZ-44701183704	active	100	
23	6056230437	44701183705	12345KU_23	6056230437_VNAMZ-44701183705	active	100	
24	6056230438	44701183706	12345KU_24	6056230438_VNAMZ-44701183706	active	100	
25	6056230439	44701183707	12345KU_25	6056230439_VNAMZ-44701183707	active	100	
26	6056230440	44701183708	12345KU_26	6056230440_VNAMZ-44701183708	active	100	
27	6056230441	44701183709	12345KU_27	6056230441_VNAMZ-44701183709	active	100	
28	6056230442	44701183710	12345KU_28	6056230442_VNAMZ-44701183710	active	100	
29	6056230443	44701183711	12345KU_29	6056230443_VNAMZ-44701183711	active	100	

Figure 5.6: Experimental Results: Data Processing and Storage

Figures above illustrate the experimental results of the data processing and storage stage in one of the database tables, namely `lazada_products` and `lazada_skus`. From the figure, it can be observed that the data has been successfully processed and fully stored in the database as expected.

5.2.3 Data Visualization Experiment

After successfully deploying the ETL pipeline, we developed several dashboards using Apache Superset to visualize key business indicators of a sports retail store operating

on the Lazada e-commerce platform. Through three main dashboards, several preliminary insights can be drawn.



Figure 5.7: Store overview

The first dashboard, "**Store Overview**", shows that the store is managing approximately 15 million SKUs, indicating a large-scale operation with a diverse product catalog. Analysis of product statuses reveals that around **70%** of the products are currently *Active*, approximately **20%** are *Inactive*, and about **10%** have been *Deleted*. This suggests that the majority of the inventory is actively listed and available for purchase, although a notable proportion of items are either paused or removed. The product distribution by category indicates a concentration in a few main categories, possibly representing the store's core product lines.

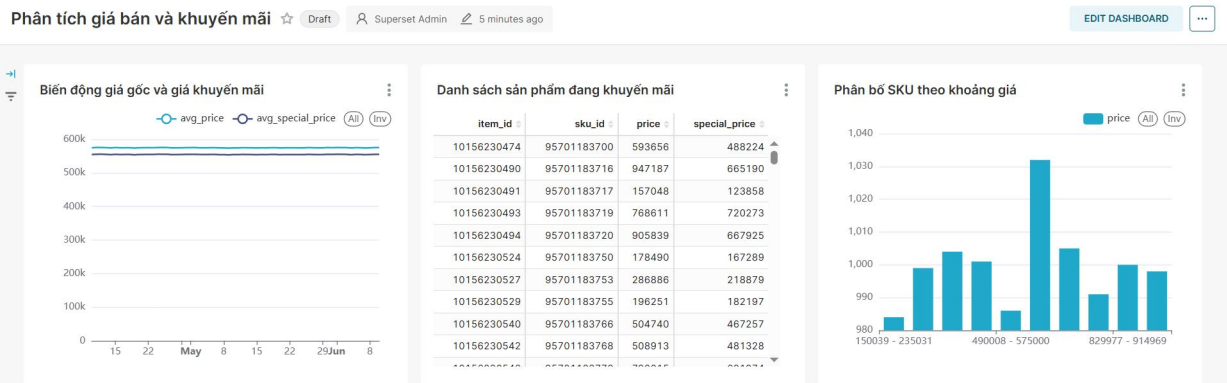


Figure 5.8: Pricing and promotion analysis

The second dashboard, titled "**Pricing and Promotion Analysis**", illustrates the price trends over time. Average product prices remain relatively stable, fluctuating around 550,000 to 600,000 VND. Promotional pricing is consistently lower than the regular price, reflecting an ongoing discount strategy. A detailed table of discounted items allows managers to quickly identify which SKUs are under promotion. Furthermore, the

histogram of SKU distribution by price range shows a strong focus on mid-range products (150,000 to 900,000 VND), with certain price bands containing a disproportionately high number of products.



Figure 5.9: Inventory and supply analysis

The third dashboard, **"Inventory and Supply Analysis"**, provides a comprehensive view of the store's inventory capabilities. The total quantity in stock reaches nearly 3 billion units, demonstrating extensive storage and supply capacity. The amount of available inventory remains consistently high (close to 40 million units) with minimal fluctuation, suggesting a stable supply chain and effective inventory forecasting. Notably, the number of out-of-stock SKUs is zero at the time of observation, indicating efficient stock management.

In summary, these Superset dashboards offer intuitive and comprehensive visualizations that aid store managers in monitoring operations, making informed decisions, and detecting anomalies in real time.

5.2.4 System Monitoring Experiment

To ensure stability, performance, and observability throughout the data pipeline, monitoring solutions were integrated using Prometheus and Grafana. These tools provide real-time visualization and tracking of metrics from multiple components including Apache Kafka, Apache Spark, and Cassandra.

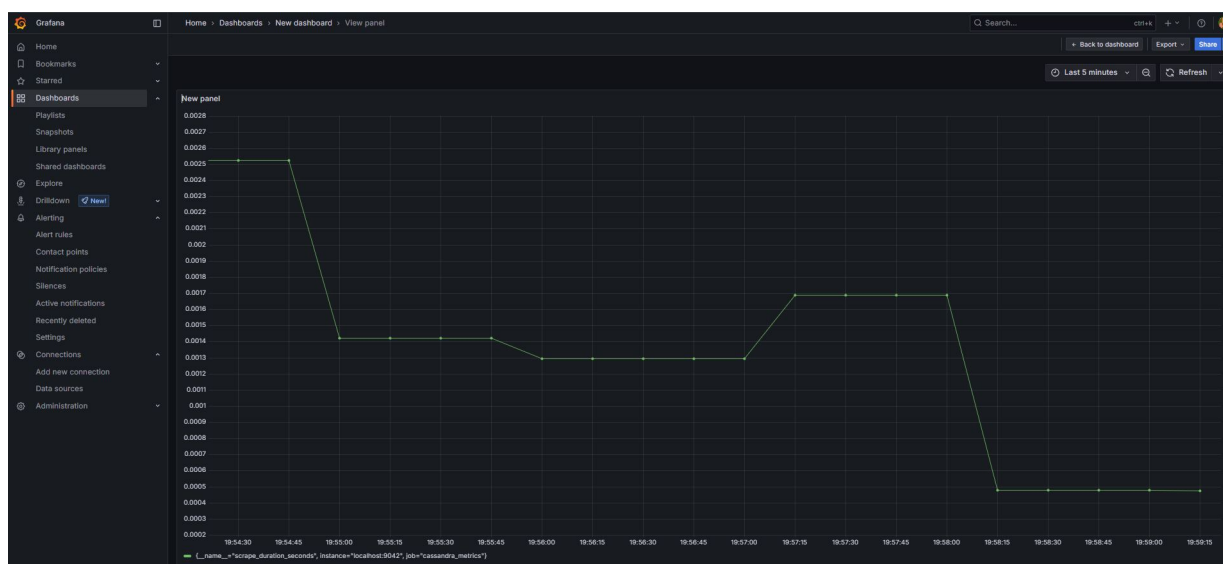


Figure 5.10: Cassandra Metrics Monitoring

As shown in Figure 5.10, metrics such as `scrape_duration_seconds` from the Cassandra node (port 9042) were successfully scraped and visualized. This metric indicates the duration Prometheus takes to collect metrics from the Cassandra exporter, and it reflects the health and responsiveness of the Cassandra node. From the observed graph, Cassandra remained responsive and stable during the monitoring window.

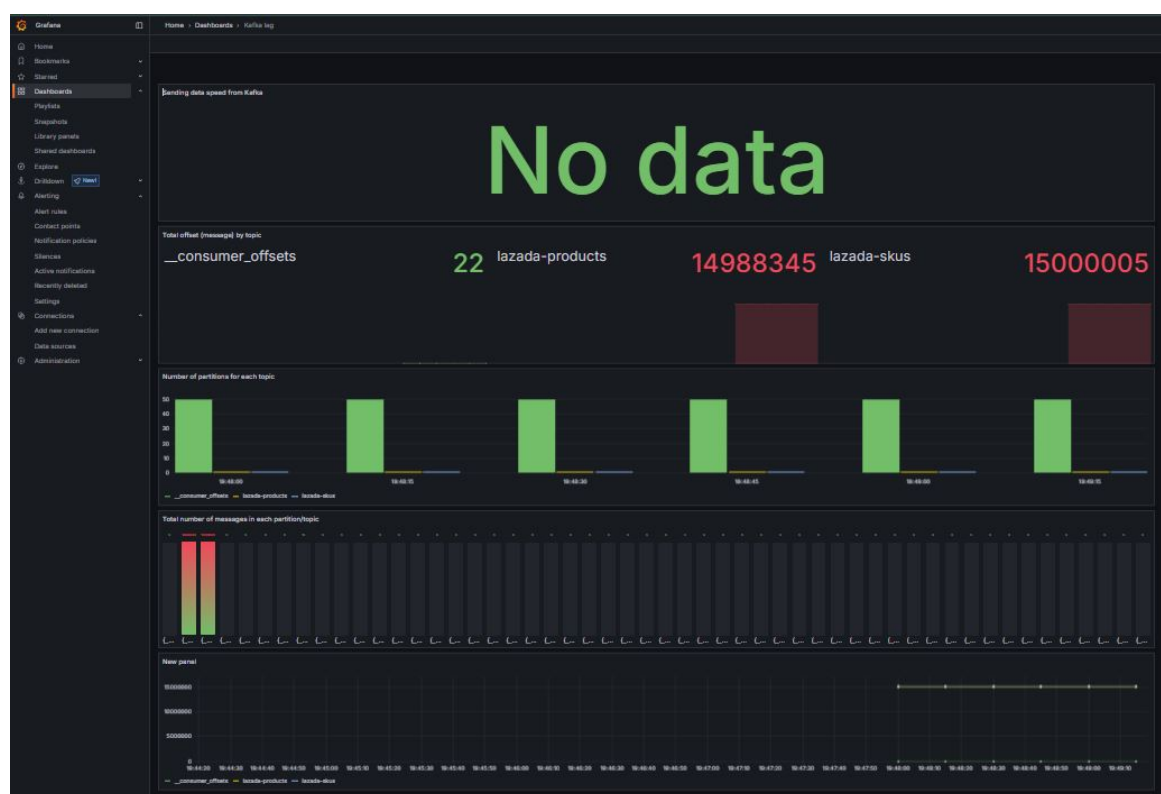


Figure 5.11: Kafka Monitoring Dashboard

Figure 5.11 illustrates a Kafka dashboard tracking various operational metrics:

- The total number of messages in topics such as `lazada-products`, `lazada-skus`, and `__consumer_offsets`.
- The number of partitions per topic.
- Real-time offset tracking, which helps evaluate how far behind the consumers are in processing the messages.

The data indicates that the system successfully consumed over 14 million messages for the `lazada-products` topic and more than 15 million messages for `lazada-skus`. This confirms the correct configuration of the producer-consumer pipeline and partitioning strategy.

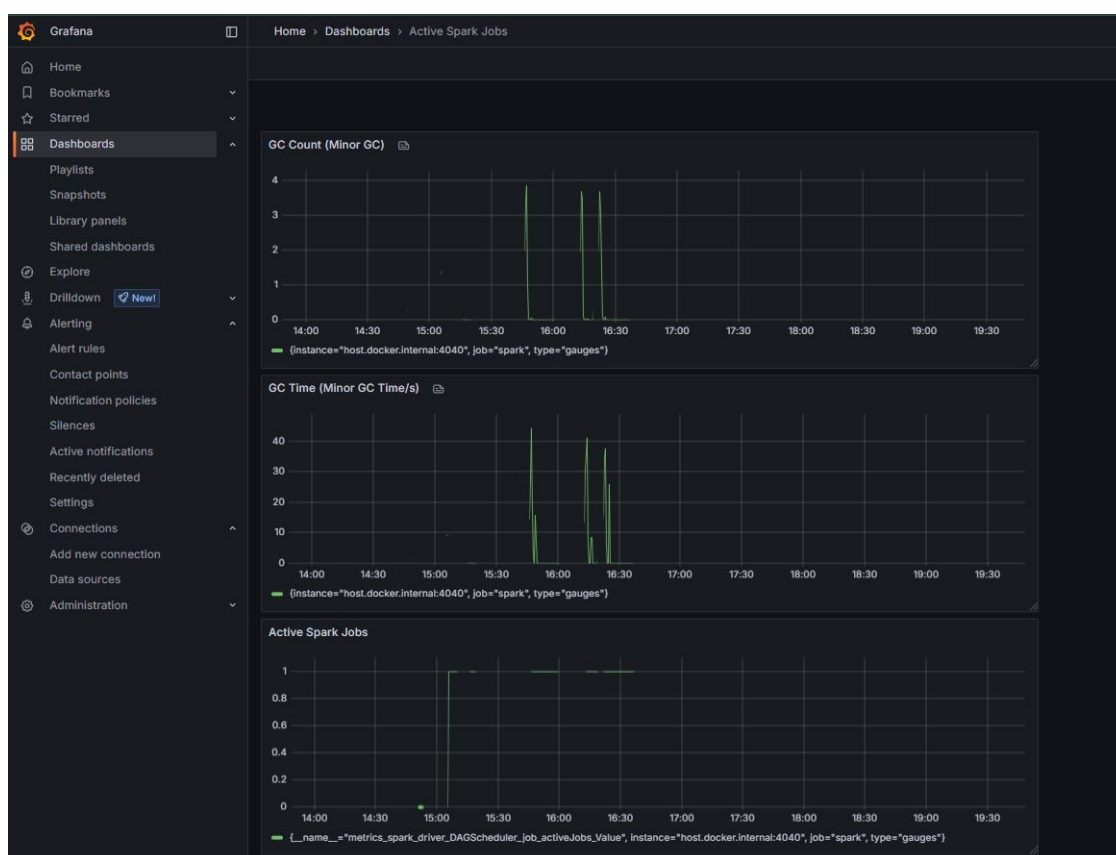


Figure 5.12: Spark Monitoring Dashboard

As shown in Figure 5.12, the Spark metrics include:

- GC Count and GC Time, which represent minor garbage collection activities.
- The number of active DAG scheduler jobs in Spark, providing insight into task execution status.

These metrics help identify performance bottlenecks or memory issues in Spark Structured Streaming jobs. During the monitored period, Spark showed multiple GC events but maintained low active job counts, which indicates efficient resource management.

To enhance system observability and automate anomaly detection, a set of custom **Prometheus alerting rules** was defined. As shown in Figure 5.13, two rule groups were created:

- **lazada_price_changes**: Detects rapid or irregular price movements.
 - `PriceVolatility`: Triggers when a product’s price fluctuates beyond a defined threshold.
 - `TechnicalIndicatorAlert`: Uses statistical conditions (e.g., moving averages) to detect abnormal trends.
- **lazada_product_alerts**: Focuses on inventory and pricing conditions.
 - `ProductPriceSpike`: Alerts when a sudden spike in product price is observed.
 - `LowInventory`: Warns when available inventory drops below a critical level.

All alert rules were configured to run every 15 seconds, with a response time under 3ms. The rules are defined in `.yaml` files and actively monitored by Prometheus, ensuring real-time awareness of critical data anomalies.

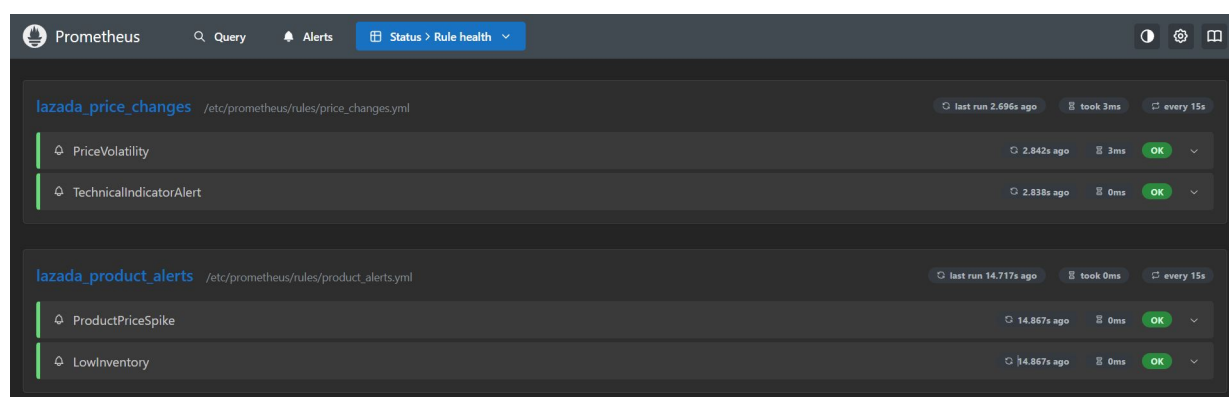


Figure 5.13: Prometheus alert rules

5.3 System Evaluation

This section presents the test scenarios designed to validate the functionality and performance of the system.

Test Scenario	Execution Steps	Expected Result	Conclusion
Testing the Data Ingestion Layer	Add a new product category and new product listings to the Lazada data source	The system successfully collects complete product data from the new category and sends it to Kafka	Success
Testing the Processing and Storage Layer	Create a test file and send valid product data to the Kafka Broker	The data is correctly processed and stored in the proper format	Success
Stopping a container in the Zookeeper/Broker cluster	During system operation, stop one container in the Zookeeper/Kafka cluster	The system continues to collect product data normally	Success
Stopping a spark-worker container	During real-time processing, stop one spark-worker container	Data processing is paused (standby mode) until the cluster recovers or reassigns the job	Partial Failure

Table 5.1: System Testing Scenarios for Lazada Data Pipeline

Table 5.1 summarizes the system testing scenarios. In the following section, I will present the results of the system’s performance testing.

Process	Time Consumption
Collecting and sending data to Kafka	60 seconds – 70 seconds
Processing and storing data	30 seconds – 40 seconds
Starting the entire system	15 seconds – 20 seconds
Response time during user interaction with the interface	0.5 seconds – 1 second

Table 5.2: System Performance Measurements

Throughout this chapter, I have presented comprehensive testing scenarios and performance experiments conducted on the developed system. The rigorous testing process helped identify and address several shortcomings, allowing iterative improvements that enhanced system quality and stability. As a result, the final product demonstrates both functional completeness and operational reliability.

Based on the presented results, it is evident that the system has successfully fulfilled the requirements defined in Chapter 2. Specifically:

From a functional perspective, the system accurately collects product data from the Lazada Open Platform API, transforms and cleanses it according to business logic,

stores the data in an optimized PostgreSQL schema, and presents it through rich visualizations using Grafana dashboards.

From a non-functional perspective, the system achieves fast processing times even under high data volume, supports horizontal scalability when integrating additional data sources, maintains high reliability through fault-tolerant mechanisms, and offers an intuitive, user-friendly interface. Role-based access control ensures proper security across user tiers.

In terms of operational aspects, the system has been verified to run stably in a real-world environment. It is easy to monitor and maintain, recovers quickly from faults, and includes thorough user documentation to support onboarding and maintenance.

The performance evaluation further confirms the system's ability to handle demanding workloads: the pipeline processes thousands of product records daily; dashboard response times consistently remain under two seconds; and the system is capable of automatic recovery in the event of container or service failure.

The system has been successfully deployed and is ready for production use. Feedback from pilot users has been overwhelmingly positive, particularly in regard to the real-time analytics features and the flexibility of the dashboard customization.

The next chapter will provide overall conclusions about the thesis, along with discussions on potential improvements and future development directions.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

Throughout the process of this graduation thesis, I have explored and studied methods and solutions for analyzing e-commerce data on the Lazada platform. I successfully built an ETL and data analysis system based on the Lambda architecture. This system can collect data from the Lazada Open API, process and clean the data using modern ETL tools, store it in PostgreSQL, and visualize it through Grafana dashboards with in-depth analytical charts.

The system provides managers with an intuitive interface that includes key business metrics such as product categories, inventory status, sales performance, pricing strategies, and marketing effectiveness. This enables managers to gain a comprehensive view of brand performance and make timely, data-driven business decisions. The system also meets requirements for real-time data updates, scalability for integrating new data sources, and fault tolerance during incidents. These features are particularly valuable for future expansion and adaptation as data volumes grow and analytical demands become more complex.

While the system meets the basic project requirements, there are several limitations that need improvement in future iterations:

1. **Data Limitations:** Data collection is constrained by the quota of Lazada's Open API, which limits access to complete historical data for long-term trend analysis.
2. **Infrastructure:** Deploying multiple containers and services on a single server over extended periods affects system performance. A better container orchestration solution is needed.
3. **Dashboard Interface:** The current dashboards only include basic charts. More advanced analytical visualizations and complex KPI indicators should be added.
4. **Error Handling:** The system needs enhanced error handling mechanisms, including retries, circuit breakers, and comprehensive monitoring.

Through this project, I have gained valuable knowledge in data engineering and e-commerce analytics. I have practiced with modern technologies such as ETL pipelines, PostgreSQL, Grafana, and applied the Lambda architecture in system design. Additionally, the process has significantly improved my self-learning, research, problem-solving, and pressure-handling skills. These lessons and experiences will serve as a solid found-

dation for my future career.

6.2 Future Work

Based on the current capabilities and limitations of the system, several future development directions are proposed to enhance functionality, scalability, and usability.

1. Data Source Expansion and Collection Optimization

The system should be extended to support more comprehensive data sources. Upgrading to Lazada's Open API Business Tier will provide higher request quotas and longer-lived access tokens, making large-scale data collection more sustainable. Additional endpoints, such as product reviews, logistics status, and order data, should be integrated to provide deeper insights. To minimize API overload and improve responsiveness, caching mechanisms and rate-limiting logic need to be implemented. Moreover, a historical data backup layer should be designed to store snapshots periodically for long-term trend analysis.

2. System Architecture and Scalability Improvements

The current architecture can be transitioned into a microservices-based model to decouple ETL, storage, and visualization layers. Cloud deployment (e.g., on AWS or Azure) would enable auto-scaling, load balancing, and broader availability. Kubernetes orchestration can manage container lifecycles more efficiently. Additionally, implementing Redis as a caching layer and introducing message queues (such as RabbitMQ or Kafka-based task queues) would significantly improve task parallelism and processing resilience.

3. Enhanced Analytics and Visualization

To move beyond descriptive analytics, machine learning models should be incorporated for predictive and prescriptive capabilities. These may include sales forecasting, inventory optimization, anomaly detection, and sentiment analysis based on reviews. Superset dashboards should be expanded to include advanced metrics such as cohort analysis, campaign effectiveness, and competitive pricing intelligence. Current visualizations can be improved by enabling filters, drilldowns, and custom date ranges for users.

4. Monitoring, Alerting, and Reliability Enhancements

Although Prometheus currently collects system metrics and alerting rules are defined, the system still lacks proper alert notification mechanisms. Future work should integrate email or Telegram alerting via Alertmanager to ensure real-time awareness of critical anomalies. More robust monitoring, such as Spark job monitoring via port 4040

and Grafana dashboard templates, should be added. Circuit breakers, retry logic, and fallback handlers will help improve system fault tolerance. Furthermore, backup and disaster recovery procedures must be established to ensure data durability.

5. Improved User Interface and Accessibility

To make the system more user-friendly, a responsive web dashboard should be developed using React or similar frameworks. This frontend would allow users to interact with ETL status, schedule reports, receive alerts, and visualize metrics in a seamless manner. Custom dashboards and exportable reports should be made available for different user roles. Finally, exposing selected system data via REST APIs will allow for external integration with BI tools or third-party services.

6.3 Current System Issues and Priorities

Despite a working pipeline, several issues require attention before scaling further.

Performance bottlenecks currently stem from unoptimized database queries, missing caching layers, and container resource contention. There is also occasional latency in API data ingestion, particularly under high loads.

Scalability constraints are evident due to single-node deployment, absence of auto-scaling, and limited horizontal scaling capabilities. These issues hinder future growth under increasing data volume.

Reliability concerns include insufficient error handling in Spark jobs and the lack of end-to-end alerting or fallback mechanisms. No current disaster recovery strategy exists, and Prometheus alerts are not yet connected to a notification system.

Technical debt has accumulated in the form of unmodularized ETL scripts, minimal test coverage, and outdated dependencies in several components. Documentation is also incomplete, making onboarding and debugging more difficult.

Addressing these limitations will greatly improve the system's stability, scalability, and maintainability, forming a solid foundation for production-ready deployment and long-term data analytics growth.

REFERENCE

- [1] Apache Software Foundation, *Apache Kafka Documentation*. [Online]. Available: <https://kafka.apache.org/documentation/> (visited on 03/27/2025).
- [2] Softqubes Technologies, *Apache Kafka – A Comprehensive Guide to Real-Time Data Streaming and Processing*. [Online]. Available: <https://www.softqubes.com/blog/apache-kafka-a-comprehensive-guide-to-real-time-data-streaming-and-processing/> (visited on 04/25/2025).
- [3] Apache Software Foundation, *Apache ZooKeeper Documentation*. [Online]. Available: <https://zookeeper.apache.org/doc/current/> (visited on 04/21/2025).
- [4] Apache Software Foundation, *Apache Spark Documentation*. [Online]. Available: <https://spark.apache.org/docs/> (visited on 03/02/2025)
- [5] CloudxLab, *Big Data Solution – Apache Hadoop and Spark*. [Online]. Available: <https://cloudxlab.com/blog/big-data-solution-apache-hadoop-and-spark/> (visited on 06/01/2025).
- [6] Apache Software Foundation, *Apache Cassandra Documentation*. [Online]. Available: <https://cassandra.apache.org/doc/latest/cassandra/architecture/> (visited on 05/04/2025).
- [7] Apache Software Foundation, *Apache Hadoop HDFS Documentation*. [Online]. Available: <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html> (visited on 04/21/2025).
- [8] TopDev, *Nền tảng Machine Learning On-Premise: Tối Ưu Hóa Hệ Thống Dữ Liệu Doanh Nghiệp*. [Online]. Available: <https://topdev.vn/blog/nen-tang-machine-learning-on-premise/> (visited on 03/16/2025).
- [9] Apache Software Foundation, *Apache NiFi Documentation*. [Online]. Available: <https://nifi.apache.org/docs/> (visited on 04/21/2025).
- [10] Demanejar, *Apache NiFi – A Data Integration Tool for Automating Data Flow Between Systems*. [Online]. Available: <https://demanejar.github.io/posts/Apache-Nifi/> (visited on 05/16/2025).
- [11] Docker Inc., *Docker Documentation*. [Online]. Available: <https://docs.docker.com/> (visited on 03/03/2025).
- [12] Purohit Sanket, *How to Backup a MySQL Database using Node.js*. [Online].

Available: <https://medium.com/@purohitsanket08/how-to-backup-a-mysql-database-using-node-js-d30a50eb2599> (visited on 05/16/2025).

[13] Prometheus Authors, *Prometheus Documentation*. [Online]. Available: <https://prometheus.io/docs/> (visited on 03/03/2025).

[14] Grafana Labs, *Grafana Documentation*. [Online]. Available: <https://grafana.com/docs/> (visited on 04/15/2025).

[15] Apache Software Foundation, *Apache Superset Documentation*. [Online]. Available: <https://superset.apache.org/docs/> (visited on 03/27/2025).

[16] Databricks, *Lambda Architecture Basics*. [Online]. Available: <https://www.databricks.com/glossary/lambda-architecture> (visited on 05/04/2025).

[17] Talend, *The Lambda Architecture and Big Data Quality*. [Online]. Available: <https://medium.com/@Talend/the-lambda-architecture-and-big-data-quality-49f34a11c823> (visited on 06/10/2025).

[19] Lazada Group, *Lazada Open Platform API Reference*. [Online]. Available: <https://open.lazada.com/apps/doc/api> (visited on 04/17/2025).

[18] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*. [Online]. Available: <https://www.postgresql.org/docs/> (visited on 03/27/2025).

[20] GeeksForGeeks, *Components Of Apache Spark*. [Online]. Available: <https://www.geeksforgeeks.org/java/components-of-apache-spark> (visited on 05/04/2025).